

AceSpec: An Asymmetric Edge-Cloud Collaborative Framework for Communication-Efficient LLM Inference

Yida Zhang

*School of Computer and Communication Engineering
University of Science and Technology Beijing
Beijing, China
ustbzyd@163.com*

Zhiyong Gao

*School of Computer and Communication Engineering
University of Science and Technology Beijing
Beijing, China
gzy@xs.ustb.edu.cn*

Shuaibing Yue

*School of Computer and Communication Engineering
University of Science and Technology Beijing
Beijing, China
m202421086@xs.ustb.edu.cn*

Jie Li

*School of Computer and Communication Engineering
University of Science and Technology Beijing
Beijing, China
m202521017@xs.ustb.edu.cn*

Rui Wang

*School of Computer and Communication Engineering
University of Science and Technology Beijing
Beijing, China
wangrui@ustb.edu.cn*

Abstract—Deploying Large Language Models (LLMs) on edge devices typically relies on model compression or split inference. However, compression degrades reasoning capabilities, while split inference suffers from severe Wide Area Network (WAN) communication bottlenecks. Edge-cloud speculative decoding emerges as a promising alternative, leveraging an edge small model to draft tokens for cloud verification. Yet, over volatile WANs, inevitable prediction rejections trigger catastrophic pipeline stalls and network-wide rollbacks, neutralizing collaborative gains. To overcome this, we propose AceSpec, an asymmetric edge-cloud collaborative framework. AceSpec utilizes un-saturated edge compute to proactively construct a probabilistic state cache, effectively transforming network-wide pipeline flushes into $O(1)$ local memory lookups. To preserve bandwidth, it employs an asymmetric communication protocol that transmits minimal main-chain indices uplink and compact sparse distributions downlink. Furthermore, we introduce a network-aware, Lagrangian-optimized resource allocation strategy that dynamically maximizes the local cache hit rate. Evaluations demonstrate that AceSpec achieves up to a $3.52\times$ throughput speedup and exhibits exceptional bandwidth immunity, sustaining near-peak inference performance even under severely constrained 50 Kbps WAN conditions.

Index Terms—LLM Inference, Edge-Cloud, Speculation decoding.

I. INTRODUCTION

Deploying Large Language Models (LLMs) directly at the network edge is increasingly necessary to support applications requiring advanced reasoning. This edge-native approach minimizes end-to-end interaction latency, enables real-time responsiveness. However, the high memory and computational

requirements make pure on-device execution impractical for resource-constrained edge nodes. To address this, traditional methods primarily rely on model compression or edge-cloud split inference. Unfortunately, aggressive compression techniques inevitably degrade the reasoning and generative capabilities of the model [1]. Split inference partitions the neural network across the edge and cloud, but transmitting large intermediate hidden states over Wide Area Networks (WANs) introduces a “Communication Wall”. Under low-bandwidth and high-latency WAN conditions, this heavy transmission payload outweighs the computational acceleration, making split inference inefficient [2].

To avoid the heavy tensor transmission of split inference, speculative decoding [3] has emerged as a promising alternative, compressing the communication payload from continuous dense tensors to discrete token sequences. By using a lightweight Small Language Model (SLM) at the edge to draft candidate tokens for parallel verification by a cloud-hosted target LLM, this paradigm significantly reduces data volume. However, vanilla speculative decoding was fundamentally designed for intra-node environments or server clusters equipped with high-speed interconnects (e.g., NVLink or PCIe), implicitly assuming negligible communication overhead. Consequently, when directly deployed over high-latency Wide Area Networks (WANs) in edge-cloud collaborative settings, its synchronous “stop-and-wait” dependency leads to severe communication bottlenecks [4]. To bridge this gap and hide the long WAN Round-Trip Time (RTT), recent frame-

AceSpec: 非对称边缘云协作框架

C高效沟通的LLM推理

张亿达

School of Computer and Communication Engineering
University of Science and Technology Beijing 中国
北京 ustbzyd@163.com

高志勇

School of Computer and Communication Engineering
University of Science and Technology Beijing 中国
北京 gzy@xs.ustb.edu.cn

岳帅兵

School of Computer and Communication Engineering
University of Science and Technology Beijing 中国
北京 m202421086@xs.ustb.edu.cn

李杰

School of Computer and Communication Engineering
University of Science and Technology Beijing 中国
北京 m202521017@xs.ustb.edu.cn

王锐

School of Computer and Communication Engineering
University of Science and Technology Beijing 中国
北京 wangrui@ustb.edu.cn

Abstract- 在边缘设备上部署大语言模型 (LLM) 通常依赖于模型压缩或分割推理。然而，压缩会降低推理能力，而分割推理会受到严重的广域网 (WAN) 通信瓶颈的影响。边缘云推测解码成为一种有前途的替代方案，利用边缘小模型来起草用于云验证的令牌。然而，在不稳定的广域网上，不可避免的预测拒绝会引发灾难性的管道停滞和网络范围内的回滚，从而抵消协作收益。为了克服这个问题，我们提出了 AceSpec，一种非对称边缘云协作框架。AceSpec 利用不饱和边缘计算主动构建概率状态缓存，有效地将网络范围的管道刷新转换为 $\mathcal{O}(1)$ 本地内存查找。为了保留带宽，它采用非对称通信协议，上行传输最小的主链索引，下行传输紧凑的稀疏分布。此外，我们引入了一种网络感知、拉格朗日优化的资源分配策略，可以动态地最大化本地缓存命中率。评估表明，AceSpec 实现了高达 $3.52\times$ 的吞吐量加速，并表现出出色的带宽抗扰度，即使在严重受限的 50 Kbps WAN 条件下也能维持接近峰值的推理性能。

Index Terms—LLM 推理、边缘云、推测解码。

一、简介

为了支持需要高级推理的应用程序，直接在网络边缘部署大语言模型 (LLM) 变得越来越必要。这种边缘原生方法最大限度地减少了端到端交互延迟，实现了实时响应。然而，高内存和计算能力

这些要求使得纯粹的设备上执行对于资源受限的边缘节点来说是不切实际的。为了解决这个问题，传统方法主要依赖于模型压缩或边云分割推理。不幸的是，激进的压缩技术不可避免地会降低模型的推理和生成能力[1]。分割推理将神经网络划分到边缘和云端，但通过广域网 (WAN) 传输大型中间隐藏状态会引入“通信墙”。在低带宽和高延迟 WAN 条件下，这种繁重的传输负载超过了计算加速，导致分割推理效率低下 [2]。

为了避免分割推理的繁重张量传输，推测性解码[3]已成为一种有前途的替代方案，将通信有效负载从连续密集张量压缩为离散令牌序列。通过在边缘使用轻量级小语言模型 (SLM) 来起草候选令牌，以便由云托管的目标 LLM 进行并行验证，该范例显著减少了数据量。然而，普通的推测性解码从根本上来说是为节点内环境或配备高速互连（例如 NVLink 或 PCIe）的服务器集群而设计的，隐含地假设通信开销可以忽略不计。因此，当在边缘云协作设置中直接部署在高延迟广域网 (WAN) 上时，其同步“停等”依赖性会导致严重的通信瓶颈[4]。为了弥补这一差距并隐藏较长的 WAN 往返时间 (RTT)，最近的帧 -

works attempt asynchronous pipelines. For instance, PicoSpec [5] decouples the edge-cloud dependency using a single-chain asynchronous pipeline, PipeSD [6] introduces token-batch scheduling to stream drafts, and E^2 -SCI [7] employs progressive lookahead concurrency with adaptive verification thresholds.

Despite mitigating partial latency, these architectures exhibit significant limitations when deployed over volatile, low-bandwidth WANs. Pipelined frameworks like PicoSpec rely on a tightly coupled, single-chain state synchronization mechanism. While PicoSpec preserves generative diversity by supporting stochastic nucleus sampling, the inherent distribution divergence between the draft and target models leads to frequent prediction rejections. Under a single-chain paradigm, these rejections cause pipeline stalls, resulting in substantial network-wide rollback penalties. In contrast, subsequent frameworks like PipeSD and E^2 -SCI dropping support for nucleus sampling. Supporting nucleus sampling traditionally necessitates transmitting vocabulary-wide draft probability distributions over the uplink, which readily saturates constrained WAN bandwidth. As a result, these newer frameworks fall back to deterministic greedy decoding. Even within this restricted scope, they introduce new inefficiencies. The batch transmission of PipeSD is highly vulnerable to early token rejections, where a single misprediction renders the unverified remainder of the batch obsolete and exacerbates bandwidth underutilization. Meanwhile, E^2 -SCI relies on frequent control signaling, which inflates protocol overhead without effectively overcoming the physical RTT lower bound. Furthermore, both frameworks require additional fine-tuning or the integration of auxiliary networks, thereby reducing their universality and deployment flexibility for off-the-shelf LLMs at the edge.

To overcome this communication barrier and provide a general, training-free inference solution over WANs, we propose AceSpec, an Asymmetric Collaborative Edge-Cloud Speculative Decoding framework. By trading the abundant parallel compute of the edge device for high WAN rollback costs, AceSpec proactively expands a probabilistic multi-branch token tree cache during the asynchronous verification window. To maximize this latency-masking efficacy, we manage the cache expansion through a two-stage analytical optimization. Network-Aware Budgeting bounds the total compute budget to hide local tree construction within the WAN RTT, and Task-Aware Shape Allocation uses a Lagrangian-optimized, non-uniform geometric decay strategy to deploy branches precisely where prediction rejections are most probable. Coupled with an asymmetric transmission protocol that uploads minimal main-chain indices and downloads sparsified target distributions upon rejection, AceSpec dynamically queries its optimized cache via local resampling. This transforms costly network-wide pipeline flushes into $\mathcal{O}(1)$ local memory lookups, eliminating rollback penalties while remaining universally applicable to any off-the-shelf LLM.

The main contributions of this paper are summarized as follows:

- We propose AceSpec, a training-free, asymmetric edge-

cloud collaborative architecture that natively supports stochastic nucleus sampling. AceSpec transforms catastrophic WAN rollbacks into $\mathcal{O}(1)$ local memory lookups without inflating uplink traffic.

- We formulate a two-level constrained optimization framework to systematically balance edge compute and network delays. It establishes a bandwidth-aware computational bound coupled with a Lagrangian-optimized, non-uniform geometric allocation strategy to strictly maximize the local state cache hit rate.
- Extensive evaluations on NVIDIA AGX Orin and A100 clusters demonstrate that AceSpec provides robust bandwidth immunity. It achieves up to a $3.52\times$ throughput speedup over traditional baselines, sustaining highly efficient generation even under severely constrained WAN conditions (down to 50 Kbps).

II. RELATED WORK

A. Speculative Decoding

Speculative decoding accelerates LLM inference by substituting serial generation with a “draft-then-verify” paradigm. Chen et al. [3] demonstrated that a modified rejection sampling mechanism can guarantee the output remains mathematically identical to the target model while providing significant speedups. SpecInfer [8] introduced a token tree verification mechanism, enabling the target model to evaluate multiple candidate branches concurrently. To enhance draft quality and throughput, Medusa [9] employs multiple parallel decoding heads fine-tuned on a frozen backbone, while EAGLE [10] uses feature-layer extrapolation to generate high-quality drafts with minimal overhead. Recent studies further optimize drafting costs and parallelization; for instance, PEARL [11] and SwiftSpec [12] explore advanced verification mechanisms and parallel tree generation, while CoSine [13] routes requests across multiple GPUs. Furthermore, SSD [14] pre-emptively prepares a speculation cache for multiple verification outcomes to eliminate local drafting latency.

However, these methods implicitly assume near-zero communication costs and are fundamentally designed for intra-node environments or datacenter servers equipped with high-speed interconnects (e.g., NVLink or PCIe). When directly applied to edge-cloud collaborative environments, they exhibit significant performance degradation. The core issue lies in the mismatch between the high-latency, bandwidth-constrained nature of Wide Area Networks (WANs) and the frequent state-synchronization required by the iterative draft-then-verify paradigm. Consequently, they cannot be deployed directly in WAN environments without exacerbating the communication bottleneck.

B. Edge-Cloud Inference

Edge-cloud collaborative inference aims to overcome the resource constraints of edge nodes through model splitting and task offloading. Frameworks like EdgeShard [15] and Jupiter [16] employ dynamic programming and intra-sequence

作品尝试异步管道。例如, PicoSpec [5] 使用单链异步管道解耦边缘-云依赖性, PipeSD [6] 引入令牌批量调度来流草稿, 而 E^2 -SCI [7] 采用具有自适应验证阈值的渐进式前瞻并发。

尽管减少了部分延迟, 但这些架构在部署在不稳定的低带宽 WAN 上时仍表现出很大的局限性。像 PicoSpec 这样的流水线框架依赖于紧密耦合的单链状态同步机制。虽然 PicoSpec 通过支持随机核采样来保留生成多样性, 但草稿模型和目标模型之间固有的分布差异会导致频繁的预测拒绝。在单链范例下, 这些拒绝会导致管道停顿, 从而导致大量的网络范围的回滚惩罚。相比之下, PipeSD 和 E^2 -SCI 等后续框架放弃了对核采样的支持。传统上, 支持核心采样需要通过上行链路传输词汇范围的草案概率分布, 这很容易使有限的 WAN 带宽饱和。因此, 这些较新的框架退回到确定性贪婪解码。即使在这个有限的范围内, 它们也会带来新的低效率。PipeSD 的批量传输很容易受到早期令牌拒绝的影响, 其中单个错误预测会导致批次中未经验证的剩余部分过时, 并加剧带宽利用率不足。同时, E^2 -SCI 依赖于频繁的控制信令, 这会增加协议开销, 而无法有效克服物理 RTT 下限。此外, 这两个框架都需要额外的微调或辅助网络的集成, 从而降低了它们对于边缘现成的 LLM 的通用性和部署灵活性。

为了克服这种通信障碍并通过 WAN 提供通用的、免训练的推理解决方案, 我们提出了 AceSpec, 一种非对称协作边缘云推理解码框架。通过以边缘设备丰富的并行计算换取高 WAN 回滚成本, AceSpec 在异步验证窗口期间主动扩展概率多分支令牌树缓存。为了最大限度地提高延迟屏蔽功效, 我们通过两阶段分析优化来管理缓存扩展。网络感知预算限制总计算预算, 以隐藏 WAN RTT 内的本地树结构, 任务感知形状分配使用拉格朗日优化的非均匀几何衰减策略, 在最有可能出现预测拒绝的地方精确部署分支。结合上传最小主链索引并在拒绝时下载稀疏目标分布的非对称传输协议, AceSpec 通过本地重采样动态查询其优化缓存。这将昂贵的网络范围管道刷新转换为 $O(1)$ 本地内存查找, 消除了回滚惩罚, 同时保持普遍适用于任何现成的 LLM。

本文的主要贡献总结如下:

- 我们提出了 AceSpec, 一种免训练、非对称边缘-

原生支持随机核采样的云协作架构。AceSpec 将灾难性的 WAN 回滚转换为 $O(1)$ 本地内存查找, 而不会增加上行链路流量。

- 我们制定了一个两级约束优化框架来系统地平衡边缘计算和网络延迟。它建立了一个带宽感知的计算界限, 加上拉格朗日优化的非均匀几何分配策略, 以严格最大化本地状态缓存命中率。
- 对 NVIDIA AGX Orin 和 A100 集群的广泛评估表明, AceSpec 提供强大的带宽抗扰度。与传统基准相比, 它的吞吐量加速高达 $3.52\times$, 即使在严重受限的 WAN 条件下 (低至 50 Kbps) 也能维持高效发电。

二.相关工作

A. Speculative Decoding

推测性解码通过用“草稿然后验证”范例代替串行生成来加速 LLM 推理。陈等人。[3] 证明, 改进的拒绝采样机制可以保证输出在数学上与目标模型相同, 同时提供显著的加速。SpecInfer [8] 引入了令牌树验证机制, 使目标模型能够同时评估多个候选分支。为了提高草稿质量和吞吐量, Medusa [9] 采用在冻结主干上微调的多个并行解码头, 而 EAGLE [10] 使用特征层外推以最小的开销生成高质量草稿。最近的研究进一步优化了绘图成本和并行化; 例如, PEARL [11] 和 SwiftSpec [12] 探索先进的验证机制和并行树生成, 而 CoSine [13] 跨多个 GPU 路由请求。此外, SSD [14] 预先为多个验证结果准备推测缓存, 以消除本地起草延迟。

然而, 这些方法隐含地假设通信成本接近于零, 并且基本上是为节点内环境或配备高速互连 (例如 NVLink 或 PCIe) 的数据中心服务器而设计的。当直接应用于边缘云协作环境时, 它们表现出显著的性能下降。核心问题在于广域网 (WAN) 的高延迟、带宽受限特性与迭代“先起草后验证”范式所需的频繁状态同步之间的不匹配。因此, 它们无法直接部署在 WAN 环境中, 而不加剧通信瓶颈。

B. Edge-Cloud Inference

边云协同推理旨在通过模型拆分和任务卸载来克服边缘节点的资源限制。EdgeShard [15] 和 Jupiter [16] 等框架采用动态编程和序列内

pipeline parallelism to optimize model splitting across distributed nodes. Other research focuses on hybrid routing: MMSL [17] uses multi-stage scheduling, while Hybrid LLM [18] and Hybrid SLM and LLM [19] route specific requests or low-confidence tokens to the cloud based on task difficulty.

Despite reducing the local computational load, these spatial or routing-based schemes fail to overcome the “Communication Wall.” The edge device must idle during remote processing and data transmission, tying the inference speed tightly to the physical limits of network RTT and restricted uplink bandwidth. Consequently, the theoretical acceleration of offloading is often offset by the communication overhead in WANs.

C. Distributed Speculative Decoding

Recent efforts have attempted to adapt speculative decoding for distributed environments. To optimize edge performance, EdgeLLM [20] employs local parallel tree generation but imposes heavy memory and computing burdens on limited hardware without considering bandwidth efficiency. Other frameworks like Venkatesha et al. [21] and HAT [22] introduce early exits or hidden-state exchanges; however, they often require invasive model modifications or inflate transmission payloads, easily saturating WAN bandwidth. To reduce uplink data, DSSD [23] moves re-sampling back to the local edge. Frameworks like DSD [24] and SLED [25] also explore dynamic windows and shared serving. Fundamentally, these architectures still adhere to a synchronous “stop-and-wait” drafting paradigm, leaving the edge idle during long WAN round trips.

To further reduce communication delays, recent frameworks propose pipelined architectures. PicoSpec [5] decouples the edge-cloud dependency using a single-chain asynchronous pipeline. PipeSD [6] overlaps token generation and communication via a token-batch scheduling mechanism, while E^2 -SCI [7] introduces progressive lookahead concurrency with adaptive verification thresholds. Despite mitigating partial network delays, these pipelines exhibit critical systemic bottlenecks in practical low-bandwidth deployments. To prevent large vocabulary-wide probability tensors from saturating the WAN uplink, they frequently fall back to deterministic greedy sampling, which prevents them from supporting the stochasticity required by real-world generative tasks. Furthermore, their reliance on single-chain drafting makes them highly vulnerable. When prediction rejections inevitably occur under nucleus sampling, the single sequence is invalidated, forcing the system to absorb substantial network-wide rollback delays. While PipeSD and E^2 -SCI attempt to bypass this, they either exacerbate bandwidth waste via obsolete batch transmissions or inflate protocol overhead, and they often require invasive model retraining. Although naive tree-based speculation could theoretically improve acceptance rates, it is generally considered unsuitable for WAN settings due to its prohibitive communication overhead [6].

These limitations underscore the need for AceSpec: a training-free, asymmetric architecture that uses available local

compute to proactively build probabilistic multi-branch caches. By utilizing optimal resource allocation to hide WAN latency, AceSpec effectively eliminates rollback penalties without inflating communication overhead.

III. METHOD

A. The Rollback Bottleneck in Edge-Cloud Inference Pipelines

To achieve efficient LLM inference on resource-constrained edge devices, edge-cloud collaboration via speculative execution is widely adopted. In this setup, an edge device uses a local Small Language Model (SLM) to generate a draft of length γ , while a cloud server performs parallel verification using a Target LLM. Let $\alpha \in (0, 1)$ denote the average single-token acceptance rate. The expected number of accepted tokens per round is $\mathbb{E}[L] = (1 - \alpha^\gamma)/(1 - \alpha)$.

In vanilla deployments, this architecture operates in a synchronous “stop-and-wait” paradigm. The cycle time is $T_{naive} = T_{draft}(\gamma) + RTT + T_{verify}$, where RTT is the Network Round-Trip Time and T_{verify} is the cloud verification time. In Wide Area Networks (WANs) where the RTT is high, this synchronous dependency creates a “Communication Wall” that leaves edge compute underutilized and significantly limits inference throughput.

To mitigate this bottleneck and maximize edge computing efficiency, existing frameworks attempt to overlap edge computation with network transmission using asynchronous pipelines. Ideally, if the cloud accepts every draft, the WAN delay is fully masked, yielding an optimal cycle time $T_{ideal} = \max(T_{draft}(\gamma), RTT + T_{verify})$. However, this approach relies on a vulnerable single-chain state synchronization mechanism. The probability of a full sequence acceptance decays exponentially ($p_{hit}^{single} = \alpha^\gamma$).

When a prediction rejection occurs, the asynchronous pipeline stalls, introducing a substantial pipeline bubble (T_{bubble}) consisting of downlink transmission, sequential edge redrafting, and uplink transmission delays. We formulate the expected time penalty incurred by these frequent network-wide pipeline flushes as:

$$\mathbb{E}[Penalty] = (1 - \alpha^\gamma) \times (RTT + T_{redraft}(\gamma)) \quad (1)$$

By incorporating this penalty, the realistic throughput of pipelined edge-cloud inference reduces to:

$$R_{pipe} = \frac{\mathbb{E}[L]}{T_{ideal} + \mathbb{E}[Penalty]} \quad (2)$$

Equation (2) highlights a fundamental limitation of current edge-cloud inference pipelines. As the acceptance rate α decreases, the penalty coefficient $(1 - \alpha^\gamma)$ approaches 1. The pipeline bubble ($RTT + T_{redraft}$) dominates the denominator, forcing the system’s cycle time to revert toward the synchronous baseline T_{naive} . Ultimately, without addressing this network rollback penalty, edge devices cannot achieve high-efficiency inference because their local compute capabilities are frequently stalled by WAN volatility.

管道并行性以优化跨分布式节点的模型分割。其他研究集中在混合路由上：MMSL [17] 使用多阶段调度，而 Hybrid LLM [18] 和 Hybrid SLM and LLM [19] 根据任务难度将特定请求或低置信度令牌路由到云。

尽管减少了本地计算负载，但这些基于空间或路由的方案未能克服“通信墙”。边缘设备在远程处理和数据传输期间必须处于空闲状态，从而将推理速度与网络 RTT 的物理限制和受限的上行链路带宽紧密联系起来。因此，理论上的卸载加速通常会被 WAN 中的通信开销所抵消。

C. Distributed Speculative Decoding

最近的努力尝试使推测解码适应分布式环境。为了优化边缘性能，EdgeLLM [20] 采用本地并行树生成，但在不考虑带宽效率的情况下，在有限的硬件上施加了沉重的内存和计算负担。其他框架如 Venkatesha 等。[21] 和 HAT [22] 引入早期退出或隐藏状态交换；然而，它们通常需要侵入性模型修改或增加传输有效负载，很容易使 WAN 带宽饱和。为了减少上行链路数据，DSSD [23] 将重采样移回本地边缘。DSD [24] 和 SLED [25] 等框架也探索动态窗口和共享服务。从根本上讲，这些架构仍然遵循同步“停等”起草范例，在长时间的 WAN 往返过程中使边缘处于空闲状态。

为了进一步减少通信延迟，最近的框架提出了流水线架构。PicoSpec [5] 使用单链异步管道解耦边缘云依赖性。PipeSD [6] 通过令牌批量调度机制重叠令牌生成和通信，而 E^2 -SCI [7] 引入了具有自适应验证阈值的渐进式前瞻并发。尽管减轻了部分网络延迟，但这些管道在实际的低带宽部署中表现出关键的系统瓶颈。为了防止大词汇范围的概率张量使广域网上行链路饱和，它们经常回退到确定性贪婪采样，这使得它们无法支持现实世界生成任务所需的随机性。此外，它们对单链起草的依赖使它们非常脆弱。当在核心采样下不可避免地发生预测拒绝时，单个序列将失效，迫使系统吸收大量的网络范围的回滚延迟。虽然 PipeSD 和 E^2 -SCI 试图绕过这个问题，但它们要么通过过时的批量传输加剧带宽浪费，要么增加协议开销，并且通常需要侵入性模型重新训练。尽管基于树的推测理论上可以提高接受率，但由于其高昂的通信开销，它通常被认为不适合 WAN 设置 [6]。

这些限制强调了对 AceSpec 的需求：一种无需训练、使用可用本地资源的非对称架构

计算以主动构建概率多分支缓存。通过利用最佳资源分配来隐藏 WAN 延迟，AceSpec 有效地消除了回滚损失，而不会增加通信开销。

三. 方法

A. The Rollback Bottleneck in Edge-Cloud Inference Pipelines

为了在资源受限的边缘设备上实现高效的 LLM 推理，通过推测执行的边缘云协作被广泛采用。在此设置中，边缘设备使用本地小语言模型 (SLM) 生成长度为 γ 的草稿，而云服务器使用目标 LLM 执行并行验证。令 $\alpha \in (0, 1)$ 表示平均单令牌接受率。每轮预计接受的代币数量为 $\mathbb{E}[L] = (1 - \alpha^\gamma)/(1 - \alpha)$ 。

在普通部署中，该架构以同步“停止并等待”模式运行。周期时间为 $T_{naive} = T_{draft}(\gamma) + RTT + T_{verify}$ ，其中 RTT 是网络往返时间， T_{verify} 是云验证时间。在 RTT 较高的广域网 (WAN) 中，这种同步依赖性会创建一道“通信墙”，导致边缘计算未得到充分利用，并显著限制推理吞吐量。

为了缓解这一瓶颈并最大限度地提高边缘计算效率，现有框架尝试使用异步管道将边缘计算与网络传输重叠。理想情况下，如果云接受每个草稿，则 WAN 延迟将被完全屏蔽，从而产生最佳周期时间 $T_{ideal} = \max(T_{draft}(\gamma), RTT + T_{verify})$ 。然而，这种方法依赖于一个脆弱的单链状态同步机制。完整序列接受的概率呈指数衰减 ($p_{hit}^{single} = \alpha^\gamma$)。

当发生预测拒绝时，异步管道停顿，引入大量管道泡沫 (T_{bubble})，其中包括下行链路传输、顺序边缘重新起草和上行链路传输延迟。我们将这些频繁的网络范围管道刷新所产生的预期时间损失表述为：

$$\mathbb{E}[Penalty] = (1 - \alpha^\gamma) \times (RTT + T_{redraft}(\gamma)) \quad (1)$$

通过纳入此惩罚，管道式边缘云推理的实际吞吐量可减少为：

$$R_{pipe} = \frac{\mathbb{E}[L]}{T_{ideal} + \mathbb{E}[Penalty]} \quad (2)$$

等式 (2) 强调了当前边缘云推理管道的基本限制。随着接受率 α 降低，惩罚系数 $(1 - \alpha^\gamma)$ 接近 1。管道气泡 ($RTT + T_{redraft}$) 主导分母，迫使系统的周期时间恢复到同步基线 T_{naive} 。最终，如果不解决网络回滚问题，边缘设备就无法实现高效推理，因为它们的本地计算能力经常因 WAN 波动而停滞。

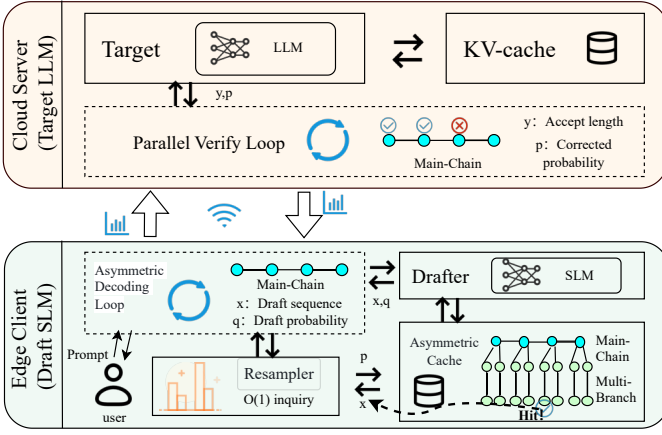


Fig. 1. The AceSpec System Architecture.

B. AceSpec

To overcome communication bottlenecks and prevent pipeline stalls, AceSpec introduces an asymmetric edge-cloud collaborative architecture. The core principle of this design is to shift the computational and decision-making burdens toward the available parallel compute of the edge, thereby minimizing the volume of data exchanged over the WAN. As illustrated in the system architecture (Fig. 1), the system comprises two primary operational entities: an Edge Client and a Cloud Server. The Edge Client hosts the lightweight draft SLM and maintains three integrated subsystems: the Tree-based Drafter, the Probabilistic KV Cache Manager, and the Local Resampler. These components interact to generate alternative speculative paths, dynamically structure the multi-branch token tree in memory, and execute non-blocking network I/O. The Cloud Server hosts the high-parameter target LLM and executes a highly parallelized Verify Loop. Rather than controlling the final token selection, the cloud acts as an evaluator that checks the validity of the edge’s proposals and generates probability distributions upon verification failures.

The execution lifecycle of AceSpec operates as a continuous, overlapping loop characterized by asynchronous data flows and localized state synchronization. To demonstrate how these architectural components interact, Fig. 2 maps out the temporal execution timeline. After completing a drafting iteration (e.g., Draft 0), the edge client executes an asymmetric stream transmission. Instead of transmitting the entire generated multi-branch tree to the cloud, it extracts only the primary linear sequence (the main chain) and sends the token indices to the server. This protocol reduces the uplink communication volume to a scalar array, conserving the constrained WAN uplink bandwidth.

After dispatching the main chain, the edge client uses the cloud turnaround time T_{cloud} as a computation window. The Tree-based Drafter performs an asynchronous multi-branch expansion starting from potential rejection points across the main chain. For instance, Draft 1 expands concurrently with the cloud’s Verify 0 stage, allowing the edge to store the

resulting state sequences in the Probabilistic Cache $\mathcal{S}$.

Concurrently, the cloud server processes the received main chain tokens in a single forward pass. If the target LLM rejects a proposed token at a specific position, the cloud server halts further validation and skips token resampling. Instead, it packages the rejection index with the corresponding partial target probability distribution for that position and returns this feedback to the edge.

Upon receiving this feedback, the Local Resampler on the edge client processes the partial target distribution and performs local token resampling to determine the correct token. This resolved token is then used as a query key (e.g., $\text{key} : \{2, 0\}$) to search the local tree cache. If the token matches a pre-computed alternative branch, the Probabilistic KV Cache Manager immediately updates the active context pointer. This local state recovery bypasses the sequential redrafting penalty, effectively hiding the WAN latency and initiating the next pipeline cycle.

C. Asymmetric Tree Construction and $\mathcal{O}(1)$ State Recovery

To implement the asynchronous pipeline introduced in Section III-B, AceSpec decouples local edge computation from network constraints using Asymmetric Branch Caching. During the cloud turnaround window (T_{cloud}), the edge SLM uses its idle compute to expand a multi-branch token tree $\mathcal{T}$. As illustrated in Fig. 3, this tree grows progressively across different decoding steps. Because the target distribution P_{k+1} of the cloud is unknown a priori, the edge uses its local draft distribution Q_{k+1} as a proxy. At each potential rejection depth $k \in \{0, \dots, \gamma - 1\}$, it excludes the main-chain token and extracts the Top- F_k candidate tokens from the residual probability mass to construct alternative branches.

To prevent drafting latency from scaling linearly with the total number of branches N , AceSpec implements a customized Tree Attention Mask. Instead of executing N sequential forward passes, all branches are packed into a single parallel forward pass. As detailed in Fig. 4, the mask comprises three distinct sub-blocks. The Shared Prefix Block allows all N branches to attend to the historically verified context $x_{1:t}$. The Main-Path Branch Block constrains a branch originating from depth s_j to attend only to the first s_j tokens of the main draft. Finally, the Suffix Diagonal Block enforces a diagonal isolation constraint so each speculative token attends exclusively to its own ancestral sequence. Consequently, the multi-branch expansion completes within an $\mathcal{O}(1)$ wall-clock duration.

The execution logic of this state recovery is formalized in Algorithm 1. After receiving the sparsified target distribution $\hat{P}_{target}$ and acceptance length π , the edge computes the corrected token $x_{resampled}$ via a local resampling operation (Line 7). The state transition occurs at Line 9. Rather than idling for a costly sequential redraft ($T_{redraft}$), the edge uses the outcome $w = (\pi, x_{resampled})$ as a query key to look up the locally cached tree $\mathcal{T}$. Upon a cache hit, it retrieves the pre-computed draft and executes a COMPACT_KV operation (Line 11). By using memory scatter/gather primitives, this

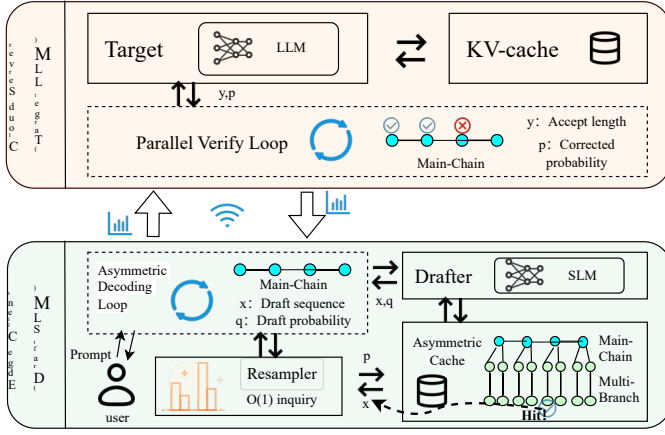


图 1. AceSpec 系统架构。

B. AceSpec

为了克服通信瓶颈并防止管道停顿，AceSpec 引入了非对称边缘云协作架构。该设计的核心原则是将计算和决策负担转移到边缘可用的并行计算上，从而最大限度地减少通过 WAN 交换的数据量。如图系统架构（图1）所示，该系统包括两个主要操作实体：边缘客户端和云服务器。边缘客户端托管轻量级草稿 SLM 并维护三个集成子系统：基于树的草稿器、概率 KV 缓存管理器和本地重采样器。这些组件交互以生成替代推测路径，在内存中动态构建多分支令牌树，并执行非阻塞网络 I/O。云服务器托管高参数目标 LLM 并执行高度并行化的验证循环。云不是控制最终的令牌选择，而是充当评估器，检查边缘提案的有效性并在验证失败时生成概率分布。

AceSpec 的执行生命周期作为一个连续的、重叠的循环运行，其特点是异步数据流和本地状态同步。为了演示这些架构组件如何交互，图 2 绘制了时间执行时间线。完成草稿迭代（例如草稿 0）后，边缘客户端执行非对称流传输。它不会将整个生成的多分支树传输到云端，而是仅提取主线性序列（主链）并将令牌索引发送到服务器。该协议将上行链路通信量减少到标量阵列，从而节省了有限的 WAN 上行链路带宽。

调度主链后，边缘客户端使用云周转时间 T_{cloud} 作为计算窗口。基于树的 Drafter 从主链上的潜在拒绝点开始执行异步多分支扩展。例如，Draft 1 与云的 Verify 0 阶段同时扩展，允许边缘存储

概率缓存 S 中的结果状态序列。

同时，云服务器在一次前向传递中处理接收到的主链代币。如果目标 LLM 在特定位置拒绝提议的令牌，云服务器将停止进一步验证并跳过令牌重新采样。相反，它将拒绝指数与该位置的相应部分目标概率分布打包，并将此反馈返回到边缘。

收到此反馈后，边缘客户端上的本地重采样器处理部分目标分布并执行本地令牌重采样以确定正确的令牌。然后，将此解析的令牌用作查询键（例如，键： $\{2, O\}$ ）来搜索本地树缓存。如果令牌与预先计算的替代分支匹配，则概率 KV 缓存管理器会立即更新活动上下文指针。这种本地状态恢复绕过了顺序重新起草惩罚，有效隐藏了 WAN 延迟并启动下一个管道周期。

C. Asymmetric Tree Construction and $\mathcal{O}(1)$ State Recovery

为了实现第 III-B 节中介绍的异步管道，AceSpec 使用非对称分支缓存将本地边缘计算与网络约束解耦。在云周转窗口 (T_{cloud}) 期间，边缘 SLM 使用其空闲计算来扩展多分支令牌树 $\mathcal{T}$ 。如图 3 所示，该树在不同的解码步骤中逐渐生长。由于云的目标分布 P_{k+1} 是先验未知的，因此边缘使用其本地草案分布 Q_{k+1} 作为代理。在每个潜在拒绝深度 $k \in \{0, \dots, \gamma - 1\}$ 处，它排除主链令牌并从剩余概率质量中提取 Top- F_k 候选令牌以构造替代分支。

为了防止起草延迟随分支总数 N 线性扩展，AceSpec 实现了定制的树注意力掩码。所有分支都被打包到单个并行前向传递中，而不是执行 N 顺序前向传递。如图 4 所述，掩模包括三个不同的子块。共享前缀块允许所有 N 分支参与历史验证的上下文 $x_{1:to}$ 。主路径分支块约束源自深度 s_j 的分支仅关注主草稿的前 s_j 标记。最后，后缀对角块强制执行对角隔离约束，因此每个推测令牌只关注其自己的祖先序列。因此，多分支扩展在 $\mathcal{O}(1)$ 挂钟持续时间内完成。

该状态恢复的执行逻辑在算法 1 中形式化。在接收到稀疏化的目标分布 $\hat{P}_{target}$ 和接受长度 π 后，边缘通过本地重采样操作计算校正后的令牌 $x_{resampled}$ （第 7 行）。状态转换发生在第 9 行。边缘不会闲置以进行代价高昂的顺序重绘 ($T_{redraft}$)，而是使用结果 $w = (\pi, x_{resampled})$ 作为查询键来查找本地缓存的树 $\mathcal{T}$ 。缓存命中后，它会检索预先计算的草稿并执行 COMPACT_KV 操作（第 11 行）。通过使用内存分散/聚集原语，这

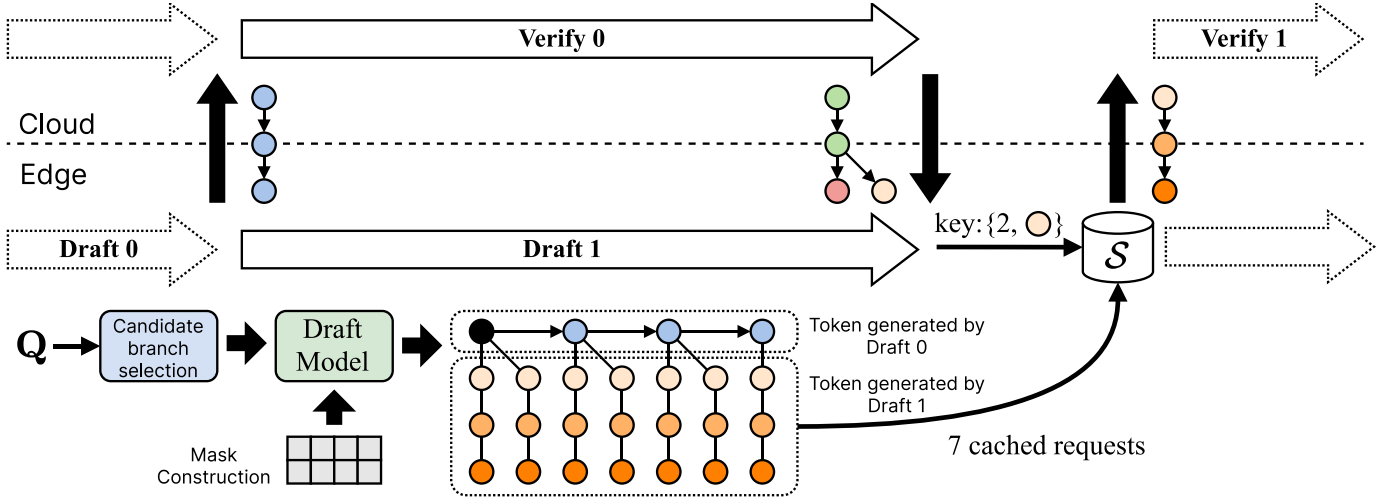


Fig. 2. The Execution Pipeline of AceSpec.

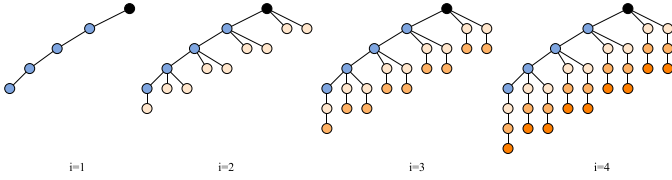


Fig. 3. The progressive growth of the multi-branch token tree across different decoding steps ($i = 1$ to 4) during the asynchronous window.

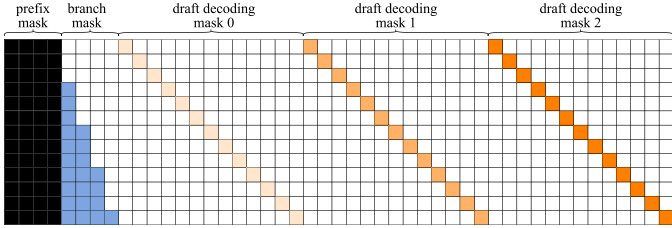


Fig. 4. The topology of the Tree Attention Mask, comprising the Shared Prefix Block, Main-Path Branch Block, and Suffix Diagonal Block.

operation aligns the KV entries of the hit branch behind the verified prefix and invalidates unselected branches. This local $\mathcal{O}(1)$ memory compaction eliminates the sequential redrafting penalty, transforming a costly pipeline flush into a zero-overhead local memory lookup.

IV. THEORETICAL OPTIMAL BRANCH ALLOCATION

A. Expected Time and Speedup Analysis of Multi-Branch Caching

To analyze the performance gain, we define the theoretical speedup S of AceSpec relative to the standard synchronous baseline. We abstract the latency components into three system constants. Let $T_{naive} = T_{draft}(\gamma) + RTT + T_{verify}$ denote the cycle time of the stop-and-wait baseline. For the pipelined architecture, we define $T_{ideal} = \max(T_{tree_draft}(B), RTT + T_{verify})$ as the ideal cycle time assuming a perfect cache hit

Algorithm 1 AceSpec Edge-Side Asynchronous Pipeline

Require: Draft SLM $\mathcal{M}_{edge}$, Draft length γ , Fan-out vector F , Input prefix X .

- 1: Initialize KV Cache and generate initial draft $\tilde{x}, Q \leftarrow \text{DRAFT}(\mathcal{M}_{edge}, X, \gamma)$
- 2: **while** generation not finished **do**
- 3: $\hat{q} \leftarrow \text{COMPRESS}(Q)$ {Asymmetric Transmission}
- 4: $\text{SEND_ASYNC}(\text{Cloud}, \langle \tilde{x}, \hat{q}, \gamma \rangle)$
- 5: $\mathcal{T} \leftarrow \text{TREE_DRAFT}(\mathcal{M}_{edge}, \tilde{x}, Q, F)$ {Asynchronous Multi-Branching}
- 6: $(\pi, \hat{P}_{target}) \leftarrow \text{RECEIVE_WAIT}(\text{Cloud})$ {Wait for verification}
- 7: $x_{resampled} \leftarrow \text{RESAMPLE}(\hat{P}_{target}, Q)$ {Local Token Resampling}
- 8: $w \leftarrow (\pi, x_{resampled})$ {Verification Outcome}
- 9: **if** $w \in \text{dom}(\mathcal{T})$ **then**
- 10: $(\tilde{x}_{next}, Q_{next}) \leftarrow \mathcal{T}[w]$ { $\mathcal{O}(1)$ State Recovery}
- 11: $\text{COMPACT_KV}(\mathcal{M}_{edge}, w)$ {Continuous memory alignment}
- 12: $\tilde{x} \leftarrow \tilde{x}_{next}; Q \leftarrow Q_{next}$
- 13: **else**
- 14: $\text{ROLLBACK_KV}(\mathcal{M}_{edge}, \gamma - \pi)$ {Cache miss penalty}
- 15: $\tilde{x}, Q \leftarrow \text{DRAFT}(\mathcal{M}_{edge}, x_{resampled}, \gamma)$ {Sequential Redrafting}
- 16: **end if**
- 17: **end while**

rate, and $T_{bubble} = RTT + T_{redraft}(\gamma)$ as the time penalty incurred upon a cache miss.

Using these variables, the expected cycle time of the multi-branch system is expressed as $\mathbb{E}[T_{tree}] = T_{ideal} + (1 - P_{hit}^{tree}(\mathcal{T}))T_{bubble}$. Because the multi-branch tree provides at least the same expected accepted length $\mathbb{E}[L]$ as a single chain, the lower bound of the speedup S is given by:

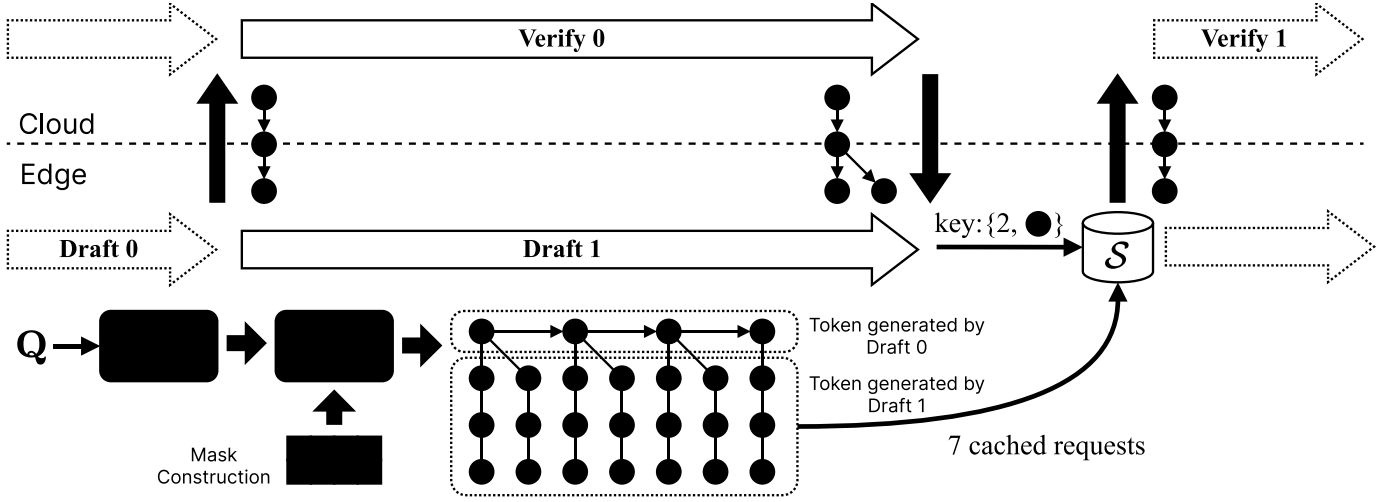


图 2. AceSpec 的执行管道。

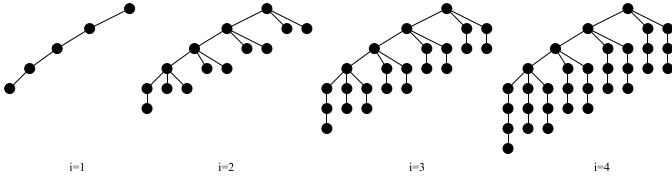


图 3. 异步窗口期间多分支令牌树在不同解码步骤 ($i = 1$ 到 4) 上的逐步增长。

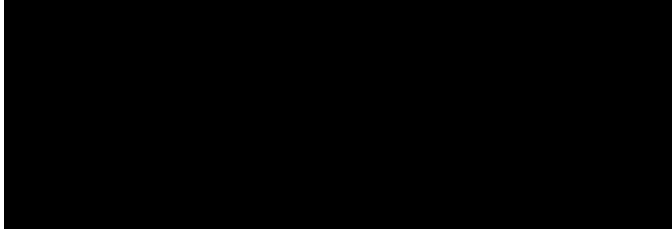


图 4. 树注意力掩模的拓扑, 包括共享前缀块、主路径分支块和后缀对角块。

操作将命中分支的 KV 条目与已验证的前缀后面对齐, 并使未选择的分支无效。这种本地 $\mathcal{O}(1)$ 内存压缩消除了顺序重新起草的损失, 将昂贵的管道刷新转变为零开销的本地内存查找。

四. 理论最优分支分配

A. Expected Time and Speedup Analysis of Multi-Branch Caching

为了分析性能增益, 我们定义了 AceSpec 相对于标准同步基线的理论加速比 S 。我们将延迟组件抽象为三个系统常量。让 $T_{naive} = T_{draft}(\gamma) + RTT + T_{verify}$ 表示停止等待基线的周期时间。对于流水线架构, 我们将 $T_{ideal} = \max(T_{tree_draft}(B), RTT + T_{verify})$ 定义为假设完美缓存命中的理想周期时间

算法 1 AceSpec 边缘侧异步管道要求: 草稿 SLM $\mathcal{M}_{edge}$ 、草稿长度 γ 、扇出向量 F 、输入前缀 X 。1: 初始化 KV 缓存并生成初始草稿 $\tilde{x}, Q \leftarrow \text{DRAFT}(\mathcal{M}_{edge}, X, \gamma)$ 2: 生成未完成时执行 3: $\hat{q} \leftarrow \text{COMPRESS}(Q)$ {非对称传输} 4: $\text{SEND ASYNC}(\text{Cloud}, \langle \tilde{x}, \hat{q}, \gamma \rangle)$ 5: $\mathcal{T} \leftarrow \text{TREE DRAFT}(\mathcal{M}_{edge}, \tilde{x}, Q, F)$ {异步多分支} 6: $(\pi, \hat{P}_{target}) \leftarrow \text{RECEIVE WAIT}(\text{Cloud})$ {等待验证} 7: $x_{resampled} \leftarrow \text{RESAMPLE}(\hat{P}_{target}, Q)$ {本地令牌重采样} 8: $w \leftarrow (\pi, x_{resampled})$ {验证结果} 9: if $w \in \text{dom}(\mathcal{T})$ 然后 10: $(\tilde{x}_{next}, Q_{next}) \leftarrow \mathcal{T}[w]$ { $\mathcal{O}(1)$ 状态恢复} 11: $\text{COMPACT KV}(\mathcal{M}_{edge}, w)$ {连续内存对齐} 12: $\tilde{x} \leftarrow \tilde{x}_{next}; Q \leftarrow Q_{next}$ 13: else 14: $\text{ROLLBACK KV}(\mathcal{M}_{edge}, \gamma - \pi)$ {缓存未命中惩罚} 15: $\tilde{x}, Q \leftarrow \text{DRAFT}(\mathcal{M}_{edge}, x_{resampled}, \gamma)$ {顺序重新起草} 16: end if 17: end while

率, $T_{bubble} = RTT + T_{redraft}(\gamma)$ 是缓存未命中时产生的时间损失。

使用这些变量, 多分支系统的预期循环时间表示为 $\mathbb{E}[T_{tree}] = T_{ideal} + (1 - P_{hit}^{tree}(\mathcal{T}))T_{bubble}$ 。由于多分支树至少提供与单链相同的预期可接受长度 $\mathbb{E}[L]$, 因此加速比 S 的下限由下式给出:

$$S = \frac{R_{tree}}{R_{naive}} \geq \frac{T_{naive}}{T_{ideal} + (1 - P_{hit}^{tree}(\mathcal{T}))T_{bubble}} \quad (3)$$

By dividing the numerator and denominator by T_{ideal} , we derive a simplified formulation:

$$S \geq \frac{S_{ideal}}{1 + (1 - P_{hit}^{tree}(\mathcal{T})) \cdot \rho} \quad (4)$$

where $S_{ideal} = T_{naive}/T_{ideal}$ represents the maximum theoretical speedup limit when the pipeline is perfectly masked, and $\rho = T_{bubble}/T_{ideal}$ represents the Rollback Penalty Ratio dictated by the WAN environment and hardware compute constraints.

Equation (4) separates the physical limits of the system from its algorithmic performance.

B. Problem Formulation with Two-Level Constraints

As established in Eq. (4), maximizing the tree cache hit rate $P_{hit}^{tree}(\mathcal{T})$ is a prerequisite for hiding latency. However, constructing $\mathcal{T}$ is bounded by two distinct constraints: network conditions and algorithmic properties. To maximize $P_{hit}^{tree}(\mathcal{T})$, we formulate a two-level constrained optimization problem.

1) *Network-Aware Budgeting*: The edge device cannot arbitrarily increase the tree size B to improve the hit rate. To ensure multi-branch drafting does not violate the ideal pipeline boundary and become a new computational bottleneck, the edge execution time must be masked by the cloud turnaround time: $T_{tree_draft}(B) \leq RTT(t) + T_{verify}$.

By continuously probing the real-time WAN delay $RTT(t)$, AceSpec derives a network-aware computation budget $B_{net}(t) = \lfloor T_{tree_draft}^{-1}(RTT(t) + T_{verify}) \rfloor$, where $T_{tree_draft}^{-1}(\cdot)$ is the inverse latency function of the edge SLM. Considering the physical VRAM and parallel capacity limit of the edge device (B_{max}), the effective budget $B^*(t)$ is dynamically bounded as:

$$B^*(t) = \min(B_{max}, \lfloor T_{tree_draft}^{-1}(RTT(t) + T_{verify}) \rfloor) \quad (5)$$

This bounding provides a compute-energy tradeoff: AceSpec increases the tree size to mitigate network fluctuations and reduces the tree during favorable network conditions to conserve edge energy.

2) *Task-Aware Shape Allocation*: Given the network-aware budget $B^*(t)$, the system must determine the topological shape of $\mathcal{T}$. In our asymmetric tree architecture, the SLM generates a main draft chain of length γ . The remaining available budget for alternative branches is thus $B_{alloc}(t) = B^*(t) - \gamma$.

Let $\mathbf{F} = [F_0, F_1, \dots, F_{\gamma-1}]$ denote the branch allocation vector, where $F_k \geq 0$ represents the number of alternative branches placed at depth k . The total allocation must satisfy the hardware constraint: $\sum_{k=0}^{\gamma-1} F_k \leq B_{alloc}(t)$.

At any depth k , the probability that the cloud's target token is captured by the edge's cache relies on the output distribution of the language model. Let α denote the primary acceptance rate of the main chain token. If the main token is rejected, the

target token is drawn from a residual distribution. Let $R(F_k)$ denote the cumulative probability that the target token falls within the top F_k alternative candidates. The hit probability at depth k is formulated as $p(F_k) = \alpha + (1 - \alpha)R(F_k)$.

Assuming conditional independence across sequence steps (a standard convention in speculative decoding analysis for mathematical tractability), the overall tree cache hit rate is the product of the hit probabilities at each depth:

$$P_{hit}^{tree}(\mathbf{F}) = \prod_{k=0}^{\gamma-1} p(F_k) = \prod_{k=0}^{\gamma-1} (\alpha + (1 - \alpha)R(F_k)) \quad (6)$$

By taking the logarithm of Eq. (6) to convert the product into a computationally tractable sum, we formulate the topological shape design as a constrained nonlinear optimization problem:

$$\max_{\mathbf{F}} \sum_{k=0}^{\gamma-1} \log(\alpha + (1 - \alpha)R(F_k)) \quad (7)$$

$$\text{s.t.} \quad \sum_{k=0}^{\gamma-1} F_k \leq B_{alloc}(t) \quad (8)$$

$$F_k \geq 0, \quad \forall k \in \{0, 1, \dots, \gamma - 1\} \quad (9)$$

Equation (7) represents the task-aware optimization goal, constrained by the network-aware limit in Eq. (8). Solving this formulation yields the optimal non-uniform geometric decay strategy.

C. Lagrangian Solution and Geometric Decay Allocation

To solve the constrained optimization problem, we must account for the sequential dependency of the auto-regressive verification. In a speculative chain, a validation event at depth k occurs only if all preceding $k - 1$ tokens have been successfully accepted. Therefore, the prefix survival probability is α^k . The effective expected log-likelihood objective is thus weighted by this survival decay:

$$\mathcal{J}(\mathbf{F}) = \sum_{k=0}^{\gamma-1} \alpha^k \log(\alpha + (1 - \alpha)R(F_k)) \quad (10)$$

To derive a closed-form solution, we use the statistical properties of Large Language Models. Empirical analyses of LLM vocabulary distributions reveal a pronounced long-tail phenomenon. The residual sampling distribution exhibits a power-law sharpness, where the cumulative probability function can be bounded by $R(F_k) \approx 1 - cF_k^{-\beta}$, with $c > 0$ as a scaling constant and $\beta > 0$ representing the power-law decay exponent.

Substituting this into the hit probability yields $p(F_k) \approx 1 - c(1 - \alpha)F_k^{-\beta}$. For high-confidence drafting, $p(F_k) \rightarrow 1$, allowing the approximation $\log p(F_k) \approx -c(1 - \alpha)F_k^{-\beta}$. We then construct the Lagrangian function $\mathcal{L}(\mathbf{F}, \lambda)$ with the multiplier $\lambda \geq 0$ for the budget constraint:

$$S = \frac{R_{tree}}{R_{naive}} \geq \frac{T_{naive}}{T_{ideal} + (1 - P_{hit}^{tree}(\mathcal{T}))T_{bubble}} \quad (3)$$

通过将分子和分母除以 T_{ideal} ，我们得出一个简化的公式：

$$S \geq \frac{S_{ideal}}{1 + (1 - P_{hit}^{tree}(\mathcal{T})) \cdot \rho} \quad (4)$$

其中 $S_{ideal} = T_{naive}/T_{ideal}$ 表示管道完全屏蔽时的最大理论加速限制，而 $\rho = T_{bubble}/T_{ideal}$ 表示由 WAN 环境和硬件计算约束决定的回滚惩罚比。

等式 (4) 将系统的物理限制与其算法性能分开。

B. Problem Formulation with Two-Level Constraints

正如方程式中所确定的那样。(4)、最大化树缓存命中率 $P_{hit}^{tree}(\mathcal{T})$ 是隐藏延迟的前提。然而，构造 $\mathcal{T}$ 受到两个不同的约束：网络条件和算法属性。为了最大化 $P_{hit}^{tree}(\mathcal{T})$ ，我们制定了一个两级约束优化问题。

1) *Network-Aware Budgeting*: 边缘设备不能随意增加树大小 B 来提高命中率。为了确保多分支绘图不会违反理想的管道边界并成为新的计算瓶颈，边缘执行时间必须被云周转时间掩盖： $T_{tree_draft}(B) \leq RTT(t) + T_{verify}$

通过不断探测实时 WAN 延迟 $RTT(t)$ ，AceSpec 得出网络感知计算预算

$B_{net}(t) = \lfloor T_{tree_draft}^{-1}(RTT(t) + T_{verify}) \rfloor$ ，其中 $T_{tree_draft}^{-1}(\cdot)$ 是边缘 SLM 的逆延迟函数。考虑到边缘设备的物理 VRAM 和并行容量限制 (B_{max})，有效预算 $B^*(t)$ 动态限制为：

$$B^*(t) = \min(B_{max}, \lfloor T_{tree_draft}^{-1}(RTT(t) + T_{verify}) \rfloor) \quad (5)$$

这种边界提供了计算能量权衡：Ace-Spec 增加树大小以减轻网络波动，并在有利的网络条件下减少树大小以节省边缘能量。

2) *Task-Aware Shape Allocation*: 给定网络感知预算 $B^*(t)$ ，系统必须确定 $\mathcal{T}$ 的拓扑形状。在我们的非对称树架构中，SLM 生成长度为 γ 的主草案链。因此，替代分支的剩余可用预算为 $B_{alloc}(t) = B^*(t) - \gamma_0$ 。

令 $\mathbf{F} = [F_0, F_1, \dots, F_{\gamma-1}]$ 表示分支分配向量，其中 $F_k \geq 0$ 表示放置在深度 k 的替代分支的数量。总分配必须满足硬件约束： $\sum_{k=0}^{\gamma-1} F_k \leq B_{alloc}(t)$ 。

在任何深度 k ，云的目标令牌被边缘的缓存捕获的概率依赖于语言模型的输出分布。令 α 表示主链代币的初次接受率。如果主令牌被拒绝，则

目标令牌是从残差分布中提取的。让 $R(F_k)$ 表示目标标记落入顶部 F_k 候选者的累积概率。深度 k 处的命中概率被公式化为 $p(F_k) = \alpha + (1 - \alpha)R(F_k)$ 。

假设跨序列步骤的条件独立性（数学易处理性的推测解码分析中的标准约定），总体树缓存命中率是每个深度的命中概率的乘积：

$$P_{hit}^{tree}(\mathbf{F}) = \prod_{k=0}^{\gamma-1} p(F_k) = \prod_{k=0}^{\gamma-1} (\alpha + (1 - \alpha)R(F_k)) \quad (6)$$

通过对方程取对数。(6) 为了将乘积转换为计算上易于处理的和，我们将拓扑形状设计表示为约束非线性优化问题：

$$\max_{\mathbf{F}} \sum_{k=0}^{\gamma-1} \log(\alpha + (1 - \alpha)R(F_k)) \quad (7)$$

$$\text{s.t.} \quad \sum_{k=0}^{\gamma-1} F_k \leq B_{alloc}(t) \quad (8)$$

$$F_k \geq 0, \quad \forall k \in \{0, 1, \dots, \gamma - 1\} \quad (9)$$

等式 (7) 代表任务感知优化目标，受到等式 (7) 中网络感知限制的约束。(8)。求解该公式可得出最佳的非均匀几何衰减策略。

C. Lagrangian Solution and Geometric Decay Allocation

为了解决约束优化问题，我们必须考虑自回归验证的顺序依赖性。在推测链中，仅当所有先前的 $k - 1$ 令牌均已成功接受时，深度 k 处的验证事件才会发生。因此，前缀生存概率为 α^k 。因此，有效预期对数似然目标由生存衰减加权：

$$\mathcal{J}(\mathbf{F}) = \sum_{k=0}^{\gamma-1} \alpha^k \log(\alpha + (1 - \alpha)R(F_k)) \quad (10)$$

为了导出封闭式解决方案，我们使用大语言模型的统计特性。对大语言模型词汇分布的实证分析揭示了明显的长尾现象。残差采样分布表现出幂律锐度，其中累积概率函数可以以 $R(F_k) \approx 1 - cF_k^{-\beta}$ 为界，其中 $c > 0$ 作为缩放常数， $\beta > 0$ 表示幂律衰减指数。

将其代入命中概率会产生 $p(F_k) \approx 1 - c(1 - \alpha)F_k^{-\beta}$ 。对于高置信度绘图， $p(F_k) \rightarrow 1$ ，允许近似对数 $p(F_k) \approx -c(1 - \alpha)F_k^{-\beta}$ 。然后，我们用预算约束的乘数 $\lambda \geq 0$ 构造拉格朗日函数 $\mathcal{L}(\mathbf{F}, \lambda)$ ：

$$\mathcal{L}(\mathbf{F}, \lambda) = \sum_{k=0}^{\gamma-1} -c\alpha^k(1-\alpha)F_k^{-\beta} - \lambda \left(\sum_{k=0}^{\gamma-1} F_k - B_{alloc}(t) \right) \quad (11)$$

By taking the partial derivative of $\mathcal{L}$ with respect to each F_k and setting it to zero ($\frac{\partial \mathcal{L}}{\partial F_k} = 0$), we obtain the optimality condition:

$$\begin{aligned} c\beta(1-\alpha)\alpha^k F_k^{-(\beta+1)} - \lambda &= 0 \\ \implies F_k^* &= \left(\frac{c\beta(1-\alpha)}{\lambda} \right)^{\frac{1}{\beta+1}} \cdot \left(\alpha^{\frac{1}{\beta+1}} \right)^k \end{aligned} \quad (12)$$

Let $\Phi(\lambda) = \left(\frac{c\beta(1-\alpha)}{\lambda} \right)^{\frac{1}{\beta+1}}$ be the depth-independent allocation base, and define the topological decay factor as $\eta = \alpha^{\frac{1}{\beta+1}}$. The closed-form solution for the optimal tree shape is given by:

$$F_k^* = \Phi(\lambda) \cdot \eta^k \quad (13)$$

Since the model acceptance rate $\alpha \in (0, 1)$ and the power-law exponent $\beta > 0$, the decay factor satisfies $\eta \in (0, 1)$. To satisfy the budget constraint $\sum_{k=0}^{\gamma-1} F_k^* = B_{alloc}(t)$, we solve for $\Phi(\lambda)$ using the geometric series sum formula, yielding the final branch allocation at depth k :

$$F_k^* = B_{alloc}(t) \cdot \frac{1-\eta}{1-\eta^\gamma} \cdot \eta^k \quad (14)$$

This formulation yields a non-uniform geometric decay strategy. It reveals a key insight: because sequential drafts have a decaying probability of verification, the system should not distribute the budget $B_{alloc}(t)$ uniformly. Instead, it allocates more branches at shallow depths ($k = 0, 1$) where the pipeline is most vulnerable to early rejections. As depth k increases, the probability of reaching that depth decreases exponentially, so the optimal resource allocation decays geometrically (η^k). This algorithm aligns the network budget constraints with model probability bounds, deploying computational redundancy where rejections are most likely to occur.

V. EVALUATION

A. Experimental Setup

We implement AceSpec and all baseline frameworks using PyTorch and evaluate them on a simulated edge-cloud testbed. The edge client runs on an NVIDIA Jetson AGX Orin (32GB), representing a typical resource-constrained but parallel-capable edge node. The cloud server is equipped with $4 \times$ NVIDIA A100 (40GB) GPUs to host the target LLMs. To replicate real-world Wide Area Network (WAN) conditions, we use the Linux Traffic Control (`tc`) utility to constrain the network throughput and inject latency, with the baseline bandwidth restricted to 100 Mbps.

To evaluate AceSpec across different model architectures and parameter scales, we select model pairs from the Qwen

and LLaMA families. Specifically, we construct the Qwen-0.6B/32B and Qwen-1.7B/32B collaborative pairs, alongside the LLaMA-1B/70B pair. The end-to-end evaluations use three datasets with varying generative complexities and token acceptance rates: GSM8K for multi-step mathematical reasoning, HumanEval for code generation, and Alpaca for general instruction-following tasks.

We compare AceSpec against five inference paradigms. The first is standard Autoregressive sequential decoding, which acts as a cloud-only baseline without edge speculation. The second is Vanilla Speculative Decoding (Vanilla Spec.), which deploys standard speculative decoding over the WAN using a synchronous “stop-and-wait” mechanism. We also evaluate Split Inference (Split Inf.), a collaborative paradigm where early model layers execute on the edge device and the remaining layers are processed on the cloud. Furthermore, we compare against two latest cloud-edge collaborative speculated decoding frameworks: DSSD [23] (Distributed Split Speculative Decoding), which offloads partial verification to the edge to reduce communication overhead, and PicoSpec [5], an asynchronous pipeline framework relying on single-chain drafting.

B. End-to-End Performance Comparison

Table I presents the throughput speedup of AceSpec and the baseline methods across various model pairs and datasets, normalized against the Autoregressive baseline. Under constrained WAN conditions, traditional collaborative paradigms such as Vanilla Speculative Decoding and Split Inference experience substantial performance degradation, frequently yielding speedup ratios below 1.00x. This phenomenon highlights the impact of the communication wall, where the transmission latency of uncompressed tensors or hidden states offsets the theoretical benefits of edge-cloud parallelization. In contrast, AceSpec achieves state-of-the-art performance across most configurations. For instance, with the LLaMA-1B and 70B model pair on the HumanEval dataset, AceSpec achieves a 3.52x throughput speedup. Similarly, using the Qwen-0.6B and 32B configuration on the GSM8K dataset, AceSpec delivers a 1.75x speedup, outperforming the single-chain asynchronous pipeline of PicoSpec (1.45x) and the distributed architecture of DSSD (0.85x). This performance margin demonstrates the efficacy of the asymmetric multi-branch caching mechanism in neutralizing the rollback penalty associated with nucleus sampling rejections.

Despite its overall performance, we observe a specific degradation with the Qwen-1.7B and 32B model pair on the HumanEval dataset, where AceSpec yields a speedup of 0.98x, slightly underperforming the cloud-only Autoregressive baseline (1.00x) and PicoSpec (1.00x). This regression stems from the interaction between the draft model’s parameter scale and the characteristics of code generation tasks. HumanEval demands rigid syntactic precision, often leading to highly peaked target distributions and frequent rejections if the draft deviates structurally. When employing a larger draft SLM (1.7B parameters) on the resource-constrained edge device,

$$\mathcal{L}(\mathbf{F}, \lambda) = \sum_{k=0}^{\gamma-1} -c\alpha^k(1-\alpha)F_k^{-\beta} - \lambda \left(\sum_{k=0}^{\gamma-1} F_k - B_{alloc}(t) \right) \quad (11)$$

通过取 $\mathcal{L}$ 对每个 F_k 的偏导数并将其设置为零 ($\frac{\partial \mathcal{L}}{\partial F_k} = 0$)，我们得到最优性条件：

$$c\beta(1-\alpha)\alpha^k F_k^{-(\beta+1)} - \lambda = 0 \\ \implies F_k^* = \left(\frac{c\beta(1-\alpha)}{\lambda} \right)^{\frac{1}{\beta+1}} \cdot \left(\alpha^{\frac{1}{\beta+1}} \right)^k \quad (12)$$

令 $\Phi(\lambda) = \left(\frac{c\beta(1-\alpha)}{\lambda} \right)^{\frac{1}{\beta+1}}$ 为与深度无关的分配基础，并将拓扑衰减因子定义为 $\eta = \alpha^{\frac{1}{\beta+1}}$ 。最佳树形状的封闭式解由下式给出：

$$F_k^* = \Phi(\lambda) \cdot \eta^k \quad (13)$$

由于模型接受率 $\alpha \in (0, 1)$ 和幂律指数 $\beta > 0$ ，因此衰减因子满足 $\eta \in (0, 1)$ 。为了满足预算约束 $\sum_{k=0}^{\gamma-1} F_k^* = B_{alloc}(t)$ ，我们使用几何级数和公式求解 $\Phi(\lambda)$ ，产生深度 k 处的最终分支分配：

$$F_k^* = B_{alloc}(t) \cdot \frac{1-\eta}{1-\eta^\gamma} \cdot \eta^k \quad (14)$$

该公式产生了非均匀几何衰减策略。它揭示了一个关键的见解：由于连续草稿的验证概率递减，系统不应该均匀分配预算 $B_{alloc}(t)$ 。相反，它会在浅层深度 ($k = 0, 1$) 分配更多分支，其中管道最容易受到早期拒绝。随着深度 k 的增加，到达该深度的概率呈指数下降，因此最优资源分配呈几何衰减 (η^k)。该算法将网络预算约束与模型概率界限保持一致，在最有可能发生拒绝的地方部署计算冗余。

五、评价

A. Experimental Setup

我们使用 PyTorch 实现 AceSpec 和所有基线框架，并在模拟边缘云测试台上对其进行评估。边缘客户端在 NVIDIA Jetson AGX Orin (32GB) 上运行，代表典型的资源受限但具有并行能力的边缘节点。云服务器配备 $4 \times$ NVIDIA A100 (40GB) GPU 来托管目标 LLM。为了复制真实的广域网 (WAN) 条件，我们使用 Linux 流量控制 (tc) 实用程序来限制网络吞吐量并注入延迟，并将基线带宽限制为 100 Mbps。

为了评估不同模型架构和参数尺度的 AceSpec，我们从 Qwen 中选择模型对

和 LLaMA 家庭。具体来说，我们构建了 Qwen-0.6B/32B 和 Qwen-1.7B/32B 协作对，以及 LLaMA-1B/70B 对。端到端评估使用三个具有不同生成复杂性和令牌接受率的数据集：用于多步骤数学推理的 GSM8K、用于代码生成的 HumanEval 以及用于一般指令跟踪任务的 Alpaca。

我们将 AceSpec 与五种推理范式进行比较。第一个是标准自回归顺序解码，它充当仅云基线，没有边缘推测。第二个是普通推测解码 (Vanilla Spec.)，它使用同步“停止并等待”机制在 WAN 上部署标准推测解码。我们还评估了分割推理 (Split Inf.)，这是一种协作范例，其中早期模型层在边缘设备上执行，其余层在云上处理。此外，我们还与两个最新的云边缘协同推测解码框架进行了比较：DSSD [23] (分布式分割推测解码)，它将部分验证卸载到边缘以减少通信开销；以及 PicoSpec [5]，一种依赖于单链起草的异步管道框架。

B. End-to-End Performance Comparison

表 I 展示了 AceSpec 的吞吐量加速以及各种模型对和数据集的基线方法，并根据自回归基线进行了标准化。在受限的 WAN 条件下，传统的协作范例（例如 Vanilla Speculative Decoding 和 Split Inference）会经历显著的性能下降，通常会产生低于 1.00 倍的加速比。这种现象凸显了通信墙的影响，其中未压缩张量或隐藏状态的传输延迟抵消了边云并行化的理论优势。相比之下，AceSpec 在大多数配置中都实现了最先进的性能。例如，通过 HumanEval 数据集上的 LLaMA-1B 和 70B 模型对，AceSpec 实现了 3.52 倍的吞吐量加速。同样，在 GSM8K 数据集上使用 Qwen-0.6B 和 32B 配置，AceSpec 提供了 1.75 倍的加速，优于 PicoSpec 的单链异步管道 (1.45 倍) 和 DSSD 的分布式架构 (0.85 倍)。这一性能裕度证明了非对称多分支缓存机制在抵消与核采样拒绝相关的回滚惩罚方面的有效性。

尽管整体性能良好，但我们在 HumanEval 数据集上观察到 Qwen-1.7B 和 32B 模型对出现了特定的退化，其中 AceSpec 的加速比为 0.98 倍，略低于仅云自回归基线 (1.00x) 和 PicoSpec (1.00x)。这种回归源于草稿模型的参数规模和代码生成任务的特征之间的相互作用。HumanEval 要求严格的句法精度，如果草稿在结构上出现偏差，通常会导致高度峰值的目标分布和频繁的拒绝。当在资源受限的边缘设备上采用较大草稿 SLM (1.7B 参数) 时，

TABLE I
END-TO-END THROUGHPUT SPEEDUP

Method	Qwen 0.6B & 32B			Qwen 1.7B & 32B			LLaMA 1B & 70B		
	GSM8K	HumanEval	Alpaca	GSM8K	HumanEval	Alpaca	GSM8K	HumanEval	Alpaca
Autoregressive	1.00× (13.89)	1.00× (13.91)	1.00× (13.82)	1.00× (13.89)	1.00 × (13.91)	1.00× (13.82)	1.00× (6.87)	1.00× (6.86)	1.00× (6.87)
Vanilla Spec.	0.57×	0.44×	0.41×	0.51×	0.49×	0.42×	1.35×	1.45×	0.81×
Split Inf.	0.54×	0.53×	0.51×	0.54×	0.53×	0.51×	0.51×	0.66×	0.65×
PicoSpec	1.45×	1.13×	1.02×	1.17×	1.00 ×	0.94×	2.51×	2.90×	2.26×
DSSD	0.85×	0.74×	0.34×	0.44×	0.32×	0.27×	1.80×	1.95×	1.27×
AceSpec (Ours)	1.75 ×	1.18 ×	1.32 ×	1.38 ×	0.98×	1.16 ×	3.29 ×	3.52 ×	3.13 ×

the local computational latency required to expand a multi-branch token tree increases. Consequently, the edge generation time begins to exceed the cloud’s turnaround time ($T_{tree_draft} > T_{cloud}$). The asynchronous pipeline becomes compute-bound at the edge rather than network-bound, causing the overhead of local multi-branch drafting to outweigh the latency-hiding benefits. In contrast, when using the lighter Qwen-0.6B model on the identical task, the system maintains a positive acceleration (1.18x) because the smaller SLM is fast enough to complete its multi-branch expansion within the asynchronous window. This comparative degradation supports the theoretical boundaries established in our Network-Aware Budgeting constraint, emphasizing that the multi-branch cache expansion must be bounded to remain hidden within the network round-trip time.

C. Robustness under Constrained Bandwidth

To evaluate the resilience of AceSpec against network volatility, we simulate a constrained and variable WAN environment. Fig. 5 illustrates the end-to-end performance of AceSpec as the available network bandwidth decreases from 100 Mbps to 10 Kbps.

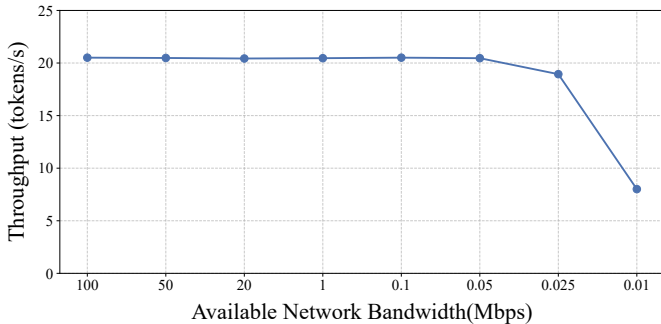


Fig. 5. End-to-end performance of AceSpec under varying network bandwidths.

AceSpec exhibits “bandwidth immunity” across various network conditions. As the bandwidth drops from 100 Mbps to 50 Kbps, the throughput remains unaffected, maintaining a stable plateau. This robustness validates the efficacy of the asymmetric transmission protocol. By transmitting only a compact array of main-chain token indices over the uplink and returning a sparsified target distribution via the downlink, AceSpec minimizes the communication payload. Consequently,

the pipeline avoids the communication bottlenecks that typically degrade traditional collaborative frameworks under sub-megabit conditions.

Performance degradation occurs only when the bandwidth is restricted to 25 Kbps and 10 Kbps. At these thresholds, the transmission delay of the minimal scalar data begins to exceed the computational latency of the edge device. Despite this drop, maintaining near-peak performance at 50 Kbps demonstrates the practical viability of AceSpec for deployment in unstable or remote edge environments.

D. Ablation Study on Branch Allocation Strategies

To evaluate the efficacy of the Lagrangian-optimized non-uniform branch allocation strategy, we conduct an ablation study comparing three drafting topologies. As presented in Table II, we evaluate a Single Branch configuration (analogous to traditional pipelined frameworks), a Uniform Multi-Branch configuration where the computational budget is distributed evenly across all depth levels, and the proposed Non-Uniform Branch configuration.

TABLE II
ABLATION STUDY ON DIFFERENT BRANCH ALLOCATION STRATEGIES.
METRICS INCLUDE TTFT (MS), DRAFT HIT RATE (%), AND THROUGHPUT (TOKENS/S).

Method	Dataset	TTFT (ms) ↓	Hit Rate ↑	Throughput ↑
Single Branch	GSM8K	442.48	0.50	20.14
	HumanEval	173.30	0.77	15.17
	Alpaca	175.14	0.23	14.09
Uniform Branch	GSM8K	450.79	0.94	21.58
	HumanEval	188.36	0.91	15.61
	Alpaca	174.01	0.74	14.24
Non-Uniform Branch	GSM8K	436.62	0.97	24.37
	HumanEval	171.33	0.97	16.57
	Alpaca	173.21	0.87	16.59

The results show that the non-uniform strategy outperforms the others across all evaluated datasets. In terms of overall generation efficiency, the non-uniform allocation achieves the highest throughput, reaching 24.37 tokens/s on the GSM8K dataset, compared to 20.14 tokens/s for the single branch and 21.58 tokens/s for the uniform topology. This gain confirms our theoretical derivation: uniformly expanding deep nodes yields diminishing marginal returns due to compounded rejection probabilities. Shifting the branch budget toward shallower, high-confidence nodes maximizes the cache hit rate under a fixed computational budget.

表 I 端到端吞吐量加速

Method	Qwen 0.6B & 32B			Qwen 1.7B & 32B			LLaMA 1B & 70B		
	GSM8K	HumanEval	Alpaca	GSM8K	HumanEval	Alpaca	GSM8K	HumanEval	Alpaca
Autoregressive	1.00× (13.89)	1.00× (13.91)	1.00× (13.82)	1.00× (13.89)	1.00× (13.91)	1.00× (13.82)	1.00× (6.87)	1.00× (6.86)	1.00× (6.87)
Vanilla Spec.	0.57×	0.44×	0.41×	0.51×	0.49×	0.42×	1.35×	1.45×	0.81×
Split Inf.	0.54×	0.53×	0.51×	0.54×	0.53×	0.51×	0.51×	0.66×	0.65×
PicoSpec	1.45×	1.13×	1.02×	1.17×	1.00×	0.94×	2.51×	2.90×	2.26×
DSSD	0.85×	0.74×	0.34×	0.44×	0.32×	0.27×	1.80×	1.95×	1.27×
AceSpec (Ours)	1.75×	1.18×	1.32×	1.38×	0.98×	1.16×	3.29×	3.52×	3.13×

扩展多分支令牌树所需的本地计算延迟增加。因此，边缘生成时间开始超过云的周转时间 ($T_{tree_draft} > T_{cloud}$)。异步管道在边缘变得受计算限制而不是受网络限制，导致本地多分支起草的开销超过了延迟隐藏的好处。相比之下，当在相同任务上使用较轻的 Qwen-0.6B 模型时，系统保持正加速度 (1.18x)，因为较小的 SLM 足够快，可以在异步窗口内完成其多分支扩展。这种相对退化支持我们的网络感知预算约束中建立的理论边界，强调多分支缓存扩展必须限制在网络往返时间内保持隐藏。

C. Robustness under Constrained Bandwidth

为了评估 AceSpec 针对网络波动的恢复能力，我们模拟了一个受限且可变的 WAN 环境。图 5 显示了当可用网络带宽从 100 Mbps 减少到 10 Kbps 时 AceSpec 的端到端性能。

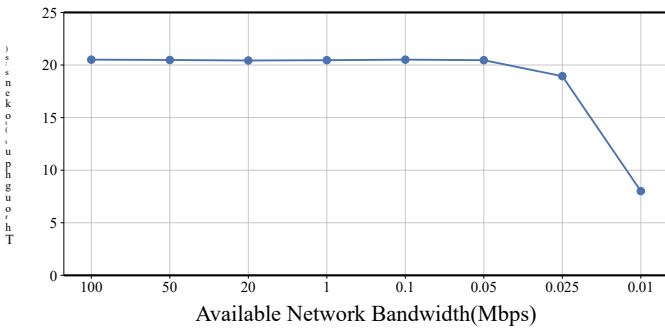


图 5. AceSpec 在不同网络带宽下的端到端性能。

AceSpec 在各种网络条件下都表现出“带宽抗扰度”。当带宽从 100 Mbps 下降到 50 Kbps 时，吞吐量不受影响，保持稳定的平台状态。这种鲁棒性验证了非对称传输协议的有效性。通过在上行链路上仅传输紧凑的主链代币索引数组并通过下行链路返回稀疏的目标分布，Ace-Spec 最大限度地减少了通信负载。最后，

该管道避免了通信瓶颈，这些瓶颈通常会在亚兆位条件下降低传统协作框架的性能。

仅当带宽限制为 25 Kbps 和 10 Kbps 时，才会出现性能下降。在这些阈值处，最小标量数据的传输延迟开始超过边缘设备的计算延迟。尽管有所下降，但仍能保持 50 Kbps 的接近峰值性能，这证明了 AceSpec 在不稳定或远程边缘环境中部署的实际可行性。

D. Ablation Study on Branch Allocation Strategies

为了评估拉格朗日优化的非均匀分支分配策略的有效性，我们进行了比较三种绘图拓扑的消融研究。如表 II 所示，我们评估了单分支配置（类似于传统的流水线框架）、统一多分支配置（其中计算预算均匀分布在所有深度级别）以及建议的非均匀分支配置。

表二不同分支分配策略的消融研究。指标包括 TTFT (MS)、草稿命中率 (%) 和吞吐量 (令牌/秒)。

Method	Dataset	TTFT (ms) ↓	Hit Rate ↑	Throughput ↑
Single Branch	GSM8K	442.48	0.50	20.14
	HumanEval	173.30	0.77	15.17
	Alpaca	175.14	0.23	14.09
Uniform Branch	GSM8K	450.79	0.94	21.58
	HumanEval	188.36	0.91	15.61
	Alpaca	174.01	0.74	14.24
Non-Uniform Branch	GSM8K	436.62	0.97	24.37
	HumanEval	171.33	0.97	16.57
	Alpaca	173.21	0.87	16.59

结果表明，在所有评估的数据集中，非均匀策略优于其他策略。在整体生成效率方面，非均匀分配实现了最高的吞吐量，在 GSM8K 数据集上达到 24.37 个令牌/秒，而单分支的 20.14 个令牌/秒和均匀拓扑的 21.58 个令牌/秒。这一增益证实了我们的理论推导：由于复合拒绝概率，均匀扩展的深度节点会产生边际收益递减。将分支预算转向较浅、高置信度的节点可在固定计算预算下最大化缓存命中率。

Analyzing the latency and hit rate components provides insights into edge compute dynamics. The Uniform Multi-Branch strategy frequently increases the Time to First Token (TTFT) (e.g., from 442.48 ms to 450.79 ms on GSM8K) because constructing a dense, uniform tree saturates the compute capacity of the edge and delays the initial dispatch of the main chain. In contrast, the Non-Uniform strategy prunes the expansion space, mitigating the initial latency overhead (achieving 436.62 ms TTFT on GSM8K) while significantly improving the draft hit rate to 0.97. We note a minor anomaly in the Alpaca dataset, where the Uniform strategy exhibits a marginally lower TTFT compared to the Single Branch baseline. Given the minimal difference of less than 2 ms, this fluctuation is primarily attributed to standard system variance and CUDA kernel launch overheads on the edge device under lightweight prompts. By concentrating the parallel capacity of the draft SLM on statistically viable alternative paths, AceSpec translates its mathematical hit-rate optimization into end-to-end acceleration.

Finally, it is worth acknowledging that the multi-branch drafting mechanism incurs certain computational overhead in its current implementation. Future optimizations, such as deploying more efficient fused attention kernels and advanced memory management, could further minimize this overhead and elevate the end-to-end inference performance.

E. Sensitivity Analysis

To understand the operational boundaries and robustness of AceSpec, we conduct a sensitivity analysis on two hyperparameters: the edge computational budget (B) and the nucleus sampling temperature (T). The draft hit rate serves as the primary metric for this analysis because it determines the efficiency of the $\mathcal{O}(1)$ state recovery.

Fig. 6 illustrates the draft hit rate as a function of the branch budget B , which represents the maximum number of alternative tokens the edge SLM explores during the asynchronous window. The results exhibit a non-linear trend: as the budget increases from 20 to 40, the hit rate rises rapidly from approximately 0.750 to over 0.850. As B scales toward 80, the growth flattens, indicating diminishing marginal returns. This phenomenon aligns with the heavy-tailed distribution of natural language; allocating a small budget captures the highly probable tokens, whereas expanding the tree further only covers the low-probability tail. This insight provides a basis for selecting a budget threshold that maximizes the hit rate while preventing compute saturation at the edge.

In Fig. 7, we analyze the resilience of the system to generation randomness by varying the sampling temperature T from 0.3 to 1.7. Elevating the temperature flattens the target distribution, increasing decoding diversity and lowering the baseline acceptance rate. Consequently, the cache hit rate declines from 0.885 at low temperatures to 0.811 at high temperatures ($T \geq 1.5$). The drop becomes steeper when the temperature exceeds 1.0. Even with this increased randomness, AceSpec maintains a hit rate above 81%. This indicates that the multi-branch cache captures a broad set of

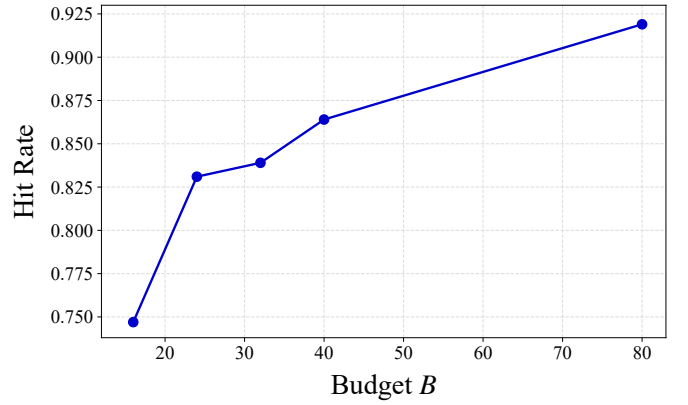


Fig. 6. Draft hit rate as a function of the branch budget B . The curve demonstrates diminishing marginal returns, guiding the allocation of edge compute.

alternative tokens, demonstrating that AceSpec is effective for both deterministic reasoning and creative text generation tasks.

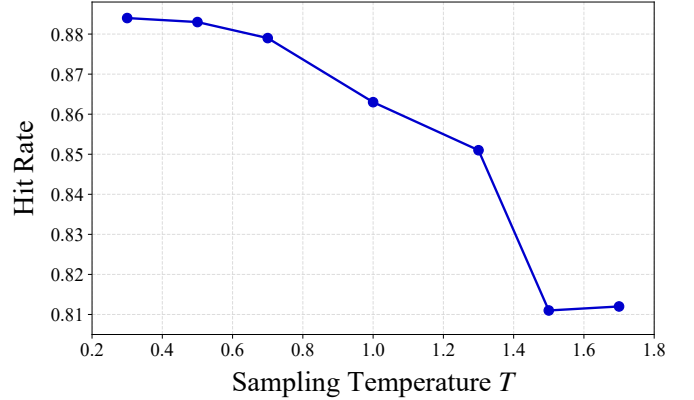


Fig. 7. Comparison of draft hit rates under different sampling temperatures T . AceSpec maintains a hit rate above 81% even in diverse generation scenarios.

VI. CONCLUSION

In this paper, we propose AceSpec, an asymmetric edge-cloud collaborative framework designed to overcome communication bottlenecks and pipeline stalls in distributed Large Language Model inference. By combining asynchronous multi-branch expansion, an $\mathcal{O}(1)$ probabilistic KV cache, and a Lagrangian-optimized non-uniform branch allocation strategy, AceSpec masks WAN latency and eliminates redrafting penalties. Extensive evaluations demonstrate that AceSpec achieves up to a $3.52\times$ throughput speedup over traditional baselines while maintaining bandwidth immunity under constrained network conditions (down to 50 Kbps). This provides an efficient, scalable, and robust solution for deploying LLMs in volatile edge environments.

REFERENCES

- [1] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on edge large language models: Design, execution, and applications,"

分析延迟和命中率组件可以深入了解边缘计算动态。统一多分支策略经常增加首次令牌时间 (TTFT) (例如, 在 GSM8K 上从 442.48 ms 增加到 450.79 ms), 因为构建密集、统一的树会使边缘的计算能力饱和并延迟主链的初始调度。相比之下, 非统一策略修剪了扩展空间, 减轻了初始延迟开销 (在 GSM8K 上实现 436.62 毫秒 TTFT), 同时将草稿命中率显着提高至 0.97。我们注意到 Alpaca 数据集中的一个小异常, 与 Single Branch 基线相比, Uniform 策略的 TTFT 略低。鉴于小于 2 毫秒的最小差异, 这种波动主要归因于标准系统差异和轻量级提示下边缘设备上的 CUDA 内核启动开销。通过将 SLM 草案的并行能力集中在统计上可行的替代路径上, AceSpec 将其数学命中率优化转化为端到端加速。

最后, 值得承认的是, 多分支起草机制在当前的实现中会产生一定的计算开销。未来的优化, 例如部署更高效的融合注意力内核和高级内存管理, 可以进一步最小化这种开销并提高端到端推理性能。

E. Sensitivity Analysis

为了了解 AceSpec 的操作边界和鲁棒性, 我们对两个超参数进行了敏感性分析: 边缘计算预算 (B) 和核采样温度 (T)。草稿命中率是此分析的主要指标, 因为它决定了 $\mathcal{O}(1)$ 状态恢复的效率。

图 6 说明了草稿命中率与分支预算 B 的函数关系, 它表示边缘 SLM 在异步窗口期间探索的替代令牌的最大数量。结果呈现出非线性趋势: 随着预算从 20 增加到 40, 命中率从大约 0.750 迅速上升到 0.850 以上。当 B 接近 80 时, 增长趋于平缓, 表明边际收益递减。这种现象与自然语言的重尾分布相一致; 分配较小的预算可以捕获高概率的标记, 而进一步扩展树只能覆盖低概率的尾部。这种见解为选择预算阈值提供了基础, 该阈值可以最大限度地提高命中率, 同时防止边缘计算饱和。

在图 7 中, 我们通过将采样温度 T 从 0.3 更改为 1.7 来分析系统对生成随机性的弹性。升高温度会使目标分布变得平坦, 增加解码多样性并降低基线接受率。因此, 缓存命中率从低温时的 0.885 下降到高温时的 0.811 ($T \geq 1.5$)。当温度超过 1.0 时, 下降变得更陡。即使随机性增加, AceSpec 仍保持高于 81% 的命中率。这表明多分支缓存捕获了广泛的数据集

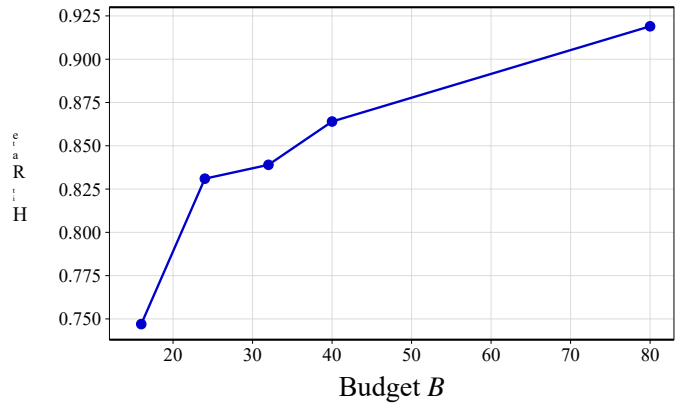


图 6. 草稿命中率与分支预算 B 的函数关系。该曲线显示边际收益递减, 指导边缘计算的分配。

替代标记, 证明 AceSpec 对于确定性推理和创造性文本生成任务都是有效的。

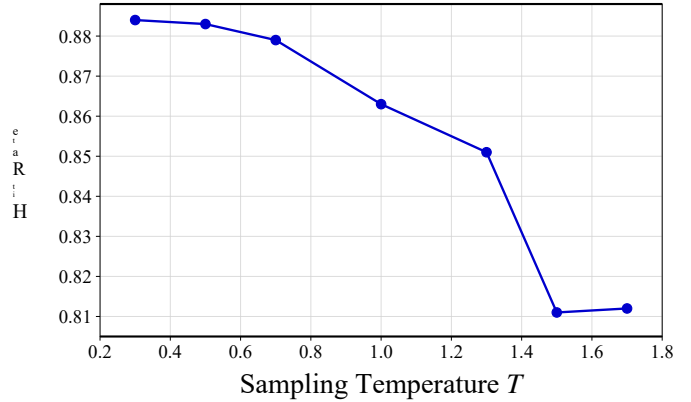


图7 不同采样温度 T 下的缓存命中率比较。即使在不同的生成场景下, AceSpec 也能保持 81% 以上的命中率。

六. 结论

在本文中, 我们提出了 AceSpec, 一种非对称边缘云协作框架, 旨在克服分布式大语言模型推理中的通信瓶颈和管道停顿。通过结合异步多分支扩展、 $\mathcal{O}(1)$ 概率 KV 缓存和拉格朗日优化的非均匀分支分配策略, AceSpec 掩盖了 WAN 延迟并消除了重新起草惩罚。广泛的评估表明, AceSpec 的吞吐量比传统基准提高了 3.52 $\times$, 同时在受限网络条件下 (低至 50 Kbps) 保持带宽抗干扰性。这为在不稳定的边缘环境中部署 LLM 提供了高效、可扩展且强大的解决方案。

参考

- [1] Y.Zheng, Y.Chen, B.Qian, X.Shi, Y.Shu 和 J.Chen, “边缘大语言模型综述: 设计、执行和应用”,

- ACM Comput. Surv.*, vol. 57, no. 8, pp. 209:1–209:35, 2025. [Online]. Available: <https://doi.org/10.1145/3719664>
- [2] R. Wang, Z. Gao, L. Zhang, S. Yue, and Z. Gao, “Empowering large language models to edge intelligence: A survey of edge efficient llms and techniques,” *Comput. Sci. Rev.*, vol. 57, p. 100755, 2025. [Online]. Available: <https://doi.org/10.1016/j.cosrev.2025.100755>
 - [3] C. Chen, S. Borgeaud, G. Irving, J. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv:2302.01318*, Feb 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.01318>
 - [4] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, Q. He, Y. Ma, M. Huang, and S. Wang, “A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 6, pp. 145:1–145:87, 2025. [Online]. Available: <https://doi.org/10.1145/3768165>
 - [5] Y. Zhang, Z. Gao, S. Yue, J. Li, and R. Wang, “A pipelined collaborative speculative decoding framework for efficient edge-cloud LLM inference,” *CoRR*, vol. abs/2603.19133, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2603.19133>
 - [6] Y. Han, Y. Gao, B. Hu, M. B. Mashhadi, Y. Duan, P. Xiao, and Y. Zhang, “Pipesd: An efficient cloud-edge collaborative pipeline inference framework with speculative decoding,” *CoRR*, vol. abs/2605.13319, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2605.13319>
 - [7] S. Li, H. Wang, Z. Jiang, Z. Jin, H. Fan, and R. Li, “E\$2\$-sci: Elastic edge-cloud speculative decoding via credit inertia,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 12 944–12 954. [Online]. Available: <https://cvpr.thecvf.com/virtual/2026/poster/36835>
 - [8] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, (ASPLOS 2024)*, vol. 3, La Jolla, CA, USA, Apr 2024, pp. 932–949. [Online]. Available: <https://doi.org/10.1145/3620666.3651335>
 - [9] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” in *Forty-first International Conference on Machine Learning, (ICML 2024)*, vol. 235, Vienna, Austria, Jul 2024, pp. 5209–5235. [Online]. Available: <https://proceedings.mlr.press/v235/cai24b.html>
 - [10] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: speculative sampling requires rethinking feature uncertainty,” in *Forty-first International Conference on Machine Learning, (ICML 2024)*, vol. 235, Vienna, Austria, Jul 2024, pp. 28 935–28 948. [Online]. Available: <https://proceedings.mlr.press/v235/li24bt.html>
 - [11] T. Liu, Y. Li, Q. Lv, K. Liu, J. Zhu, W. Hu, and X. Sun, “PEARL: parallel speculative decoding with adaptive draft length,” in *The Thirteenth International Conference on Learning Representations, (ICLR 2025)*, Singapore, April 2025. [Online]. Available: <https://openreview.net/forum?id=QOXrVMiHGK>
 - [12] Z. Zhang, Z. Jiang, C. Jiang, M. Yu, S. Zheng, H. Lin, X. Liu, and H. Hoffmann, “Swiftspec: Disaggregated speculative decoding and fused kernels for low-latency LLM inference,” in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, (ASPLOS 2026)*, vol. 2, Pittsburgh, PA, USA, Mar 2026, pp. 2197–2211. [Online]. Available: <https://doi.org/10.1145/3779212.3790246>
 - [13] L. Gao, J. Liu, H. Xu, and L. Huang, “Collaborative speculative inference for efficient LLM inference serving,” *arXiv:2503.10325*, Mar 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2503.10325>
 - [14] T. Kumar, T. Dao, and A. May, “Speculative speculative decoding,” *CoRR*, vol. abs/2603.03251, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2603.03251>
 - [15] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, “Edgeshard: Efficient LLM inference via collaborative edge computing,” *IEEE Internet Things J.*, vol. 12, no. 10, pp. 13 119–13 131, 2025. [Online]. Available: <https://doi.org/10.1109/IJOT.2024.3524255>
 - [16] S. Ye, B. Ouyang, L. Zeng, T. Qian, X. Chu, J. Tang, and X. Chen, “Jupiter: Fast and resource-efficient collaborative inference of generative llms on edge devices,” in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, London, United Kingdom, May 2025, pp. 1–10. [Online]. Available: <https://doi.org/10.1109/INFOCOM55648.2025.11044734>
 - [17] M. Ma, C. Gong, L. Zeng, and Y. Yang, “Multi-tier multi-node scheduling of LLM for collaborative AI computing,” in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, London, United Kingdom, May 2025, pp. 1–10. [Online]. Available: <https://doi.org/10.1109/INFOCOM55648.2025.11044698>
 - [18] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, and A. H. Awadallah, “Hybrid LLM: cost-efficient and quality-aware query routing,” in *The Twelfth International Conference on Learning Representations, (ICLR 2024)*, Vienna, Austria, May 2024. [Online]. Available: <https://openreview.net/forum?id=02f3mUtnqM>
 - [19] Z. Hao, H. Jiang, S. Jiang, J. Ren, and T. Cao, “Hybrid SLM and LLM for edge-cloud collaborative inference,” in *Proceedings of the Workshop on Edge and Mobile Foundation Models, EdgeFM 2024*, Minato-ku, Tokyo, Japan, Jun 2024, pp. 36–41. [Online]. Available: <https://doi.org/10.1145/3662006.3662067>
 - [20] D. Xu, W. Yin, H. Zhang, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, “Edgellm: Fast on-device LLM inference with speculative decoding,” *IEEE Trans. Mob. Comput.*, vol. 24, no. 4, pp. 3256–3273, 2025. [Online]. Available: <https://doi.org/10.1109/TMC.2024.3513457>
 - [21] Y. Venkatesha, S. Kundu, and P. Panda, “Fast and cost-effective speculative edge-cloud decoding with early exits,” *Trans. Mach. Learn. Res.*, vol. 2025, 2025. [Online]. Available: <https://openreview.net/forum?id=PTIUJARnbc>
 - [22] Z. Xie, Y. Xu, H. Xu, Y. Liao, and Z. Yao, “A novel hat-shaped device-cloud collaborative inference framework for large language models,” *arXiv:2503.18989*, Mar 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2503.18989>
 - [23] J. Ning, C. Zheng, and T. Yang, “DSSD: efficient edge-device LLM deployment and collaborative inference via distributed split speculative decoding,” *arXiv:2507.12000*, Jul 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2507.12000>
 - [24] F. Yu, L. Li, B. McDanel, and S. Q. Zhang, “DSD: A distributed speculative decoding solution for edge-cloud agile large model serving,” *arXiv:2511.21669*, Nov 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2511.21669>
 - [25] X. Li, D. Spatharakis, S. Ghafouri, J. Fan, H. Vandierendonck, D. John, B. Ji, and D. S. Nikolopoulos, “SLED: A speculative LLM decoding framework for efficient edge serving,” in *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing, SEC 2025, the Hilton Arlington National Landing*, Arlington, VA, USA, December 2025, pp. 26:1–26:8. [Online]. Available: <https://doi.org/10.1145/3769102.3770608>

ACM Comput. Surv., 卷。57、没有。8, 第 209:1–209:35, 2025 年。[在线]。可用: <https://doi.org/10.1145/3719664> [2] R. Wang, Z. Gau, L. Zhang, S. Yue 和 Z. Gau, “赋予大语言模型边缘智能: 边缘高效 llms 和技术的调查”, *Comput. Sci. Rev.*, 卷。57, p. 100755, 2025。[在线]。可用: <https://doi.org/10.1016/j.cosrev.2025.100755> [3] C. Chen, S. Borgeaud, G. Irving, J. Lespiau, L. Sifre 和 J. Jumper, “通过推测性采样加速大语言模型解码”, *arXiv:2302.01318*, 2023 年 2 月。[在线]。网址: <https://doi.org/10.48550/arXiv.2302.01318> [4] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, Q. He, Y. Ma, M. Huang 和 S. Wang, “大语言模型时代小语言模型的综合综述: 技术、增强、应用程序、与 llms 的协作以及可信度”, *ACM Trans. Intell. Syst. Technol.*, 卷。16、没有。6, 第 145:1–145:87, 2025 年。[在线]。可用: <https://doi.org/10.1145/3768165> [5] Y. Zhang, Z. Gao, S. Yue, J. Li 和 R. Wang, “一种用于高效边缘云 LLM 推理的流水线协作推测解码框架”, *CoRR*, 卷。绝对/2603.19133, 2026。[在线]。可用: <https://doi.org/10.48550/arXiv.2603.19133> [6] Y. Han, Y. Gau, B. Hu, M. B. Mashhadi, Y. Duan, P. Xiao 和 Y. Zhang, “Pipesd: 一种具有推测性解码的高效云边缘协作管道推理框架”, *CoRR*, 卷。绝对/2605.13319, 2026。[在线]。可用: <https://doi.org/10.48550/arXiv.2605.13319> [7] S. Li, H. Wang, Z. Jiang, Z. Jin, H. Fan 和 R. Li, “ES2S-sci: 通过信用惯性进行弹性边云推测解码”, 载于 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026 年 6 月, 第 12 页 944–12 954。[在线]。网址: <https://cvpr.thecvf.com/virtual/2026/poster/36835> [8] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Z. hu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar 和 Z. Ji a, “Specinfer: 加速大规模提供基于树的推测推理和验证的语言模型”, 见 *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2024)*, 卷。3, 美国加利福尼亚州拉霍亚, 2024 年 4 月, 第 932–949 页。[在线]。可用: <https://doi.org/10.1145/3620666.3651335> [9] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen 和 T. Dao, “Medusa: 具有多个解码头的简单 LLM 推理加速框架”, 第 *Forty-first International Conference on Machine Learning (ICML 2024)* 卷。235, 奥地利维也纳, 2024 年 7 月, 第 5209–5235 页。[在线]。可用: <https://proceedings.mlr.press/v235/cai24b.html> [10] Y. Li, F. Wei, C. Zhang 和 H. Zhang, “EAGLE: 推测性采样需要重新思考特征不确定性”, 第 *Forty-first International Conference on Machine Learning (ICML 2024)* 卷。235, 奥地利维也纳, 2024 年 7 月, 第 28 935–28 948 页。[在线]。可用: <https://proceedings.mlr.press/v235/li24bt.html> [11] T. Liu, Y. Li, Q. Lv, K. L. Liu, J. Zhu, W. Hu 和 X. Sun, “PEARL: 具有自适应草稿长度的并行推测解码”, *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 新加坡, 2025 年 4 月。[在线]。可用: <https://openreview.net/forum?id=QOXrVMiHGK> [12] Z. Zhang, Z. Jiang, C. Jiang, M. Yu, S. Zheng, H. Lin, X. Liu 和 H. Hoffmann, “Swiftspec: 用于低延迟 LLM 推理的分解推测解码和融合内核”, *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2026)*, 卷。2, 美国宾夕法尼亚州匹兹堡, 2026 年 3 月, 第 2197–2211 页。[在线]。可用: <https://doi.org/10.1145/3779212.3790246> [13] L. Gau, J. Liu, H. Xu 和 L. Huang, “高效大语言模型推理服务的协作推测推理”, *arXiv:2503.10325*, 2025 年 3 月。[在线]。可用: <https://doi.org/10.48550/arXiv.2503.10325> [14] T. Kumar, T. Dao 和 A. May, “推测推测解码”, *CoRR*, 卷。绝对/2603.03251, 2026。[在线]。可用: <https://doi.org/10.48550/arXiv.2603.03251> [15] M. Zhang, X. Shen, J. Cao, Z. Cui 和 S. Jiang, “Edgeshard: 通过协作边缘计算实现高效的 LLM 推理”, *IEEE Internet Things J.*, 卷。12、没有。10, 第 13 119–13 131 页, 2025 年。[在线]。可用: <https://doi.org/10.1109/IIOT.2024.3524255> [16] S. Ye, B. Ouyang, L. Zeng, T. qian, X. Chu, J. Tang 和 X. Chen, “Jupiter: 边缘设备上生成 llms 的快速且资源高效的协作推理”, *IEEE INFOCOM*

2025 - *IEEE Conference on Computer Communications*, 英国伦敦, 2025 年 5 月, 第 1-10 页。[在线]。网址: <https://doi.org/10.1109/IIOT.2024.3524255> [17] M. Ma, C. Kong, L. Zeng 和 Y. Yang, “用于协作 AI 计算的 LLM 的多层多节点调度”, *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 英国伦敦, 2025 年 5 月, 第 1-10 页。[在线]。网址: <https://doi.org/10.1109/INFOCOM55648.2025.11044734> [18] D. Ding, A. Mallick, C. Wang, R. Sim, ee, V. Rühle, L. V. S. Lakshmanan 和 A. H. Awadallah, “混合大语言模型: 成本效益和质量感知查询路由”, 载于 *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 奥地利维也纳, 2024 年 5 月。[在线]。可用: <https://openreview.net/forum?id=02f3mUtnqM> [19] Z. hao, H. Jiang, S. Jiang, J. Ren 和 T. Cao, “用于边缘云协作推理的混合 SLM 和 LLM”, *Proceedings of the Workshop on Edge and Mobile Foundation Models, EdgeFM 2024*, 日本东京港区, 2024 年 6 月, 第 36-41 页。[在线]。可用: <https://doi.org/10.1145/3662006.3662067> [20] D. Xu, W. Yin, H. Zhu, X. Jin, Y. Zhang, S. Wei, M. Xu 和 X. Liu, “Edgellm: 具有推测性解码的快速设备上 LLM 推理”, *IEEE Trans. Mob. Comput.*, 卷。24、没有。4, 第 32 56–3273 页, 2025 年。[在线]。可用: <https://doi.org/10.1109/TMC.2024.3513457> [21] Y. Venkatesha, S. Kundu 和 P. Panda, “快速且具有成本效益的推测边缘云解码并提前退出”, *Trans. Mach. Learn. Res.*, 卷。2 025, 2025。[在线]。可用: <https://openreview.net/forum?id=22jZ.Xie.Y.Xu.H.Xu.Y.Liao.Z.Yao>, “一种用于大语言模型的新型帽形设备云协作推理框架”, *arXiv:2503.18989*, 2025 3 [在线]。可用: <https://doi.org/10.48550/arXiv.2503.18989> [23] J. Ning, C. Cheng 和 T. Yang, “DSSD: 通过分布式分割推测解码实现高效边缘设备 LLM 部署和协作推理”, *arXiv:2507.12000*, 2025 年 7 月。[在线]。可用: <https://doi.org/10.48550/arXiv.2507.12000> [24] F. Yu, L. Li, B. McDanel 和 S. Q. Zhang, “DSD: 一种用于边缘云敏捷大型模型服务的分布式推测解码解决方案”, *arXiv:2511.21669*, 2025 年 11 月。[在线]。可用: <https://doi.org/10.48550/arXiv.2511.21669> [25] X. Li, D. Spatharakis, S. Ghafouri, J. Fan, H. Vandierendonck, D. John, B. Ji 和 D. S. Nikolopoulos, “SLED: 一种用于高效边缘服务的推测性 LLM 解码框架”, 参见 *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing, SEC 2025, the Hilton Arlington National Landing*, 美国弗吉尼亚州阿灵顿, 2025 年 12 月, 第 26:1–26:8。[在线]。可用: <https://doi.org/10.1145/3769102.3770608>